

```
// 源文件: src/App.jsx
import { useEffect, useMemo, useRef, useState } from "react";
import { createPortal } from "react-dom";
import {
  ArrowCounterClockwise,
  ArrowLeft,
  ArrowRight,
  ArrowSquareOut,
  Backspace,
  Bell,
  Calculator,
  CalendarBlank,
  CaretLeft,
  CaretDown,
  CaretRight,
  CaretUp,
  ChartBar,
  Check,
  CircleHalfTilt,
  Clock,
  CloudSun,
  CornersIn,
  CornersOut,
  Database,
  DownloadSimple,
  DotsThreeVertical,
  DotsSixVertical,
  FloppyDisk,
  Flag,
  FolderOpen,
  Gear,
  ListChecks,
  List,
  MapPin,
  Moon,
  MusicNotes,
  NotePencil,
  Palette,
  Pause,
  PencilSimple,
  PictureInPicture,
  Play,
  Plus,
  Repeat,
  SkipBack,
  SkipForward,
  Shuffle,
  SlidersHorizontal,
  SortAscending,
  SpeakerHigh,
```

```
SpeakerSlash,
Stop,
Sun,
Target,
TextT,
Timer,
TrendUp,
Trash,
Translate,
UploadSimple,
Wrench,
X,
} from "@phosphor-icons/react";
import {
  Hct,
  SchemeContent,
  SchemeExpressive,
  SchemeFidelity,
  SchemeFruitSalad,
  SchemeMonochrome,
  SchemeNeutral,
  SchemeRainbow,
  SchemeTonalSpot,
  SchemeVibrant,
  argbFromHex,
  hexFromArgb,
} from "@material/material-color-utilities";
import GlassSurface from "@components/GlassSurface/GlassSurface
  .jsx";
const MODES = {
  focus: { label: "专注" },
  short: { label: "短休" },
};
const TASKS_VISIBLE = 5;
const TASK_ROW_STEP = 76;
const TIMER_ORBIT_CIRCUMFERENCE = 2 * Math.PI * 48;
const CONTEXT_COMPONENTS = {
  date: { label: "日期与时间", detail: "日期、星期与当前时间" },
  weather: { label: "实时天气", detail: "定位天气或北京天气" },
  countdown: { label: "倒数日", detail: "自定义目标与剩余天数" }
  ,
  calendar: { label: "月历", detail: "当前月份、星期与今日位置"
  },
  media: { label: "当前媒体", detail: "Spotify 或本地音乐播放" }
  ,
};
const BEIJING_WEATHER_LOCATION = {
  latitude: 39.9042,
  longitude: 116.4074,
  label: "北京",
```

```
    fallback: true,
  };
  const SPOTIFY_AUTH_STORAGE_KEY = "moke-spotify-auth";
  const SPOTIFY_CLIENT_ID_STORAGE_KEY = "moke-spotify-client-id";
  const SPOTIFY_VERIFIER_STORAGE_KEY =
    "moke-spotify-code-verifier";
  const SPOTIFY_STATE_STORAGE_KEY = "moke-spotify-auth-state";
  const MEDIA_SOURCE_STORAGE_KEY = "tomatotodo-media-source";
  const LOCAL_MEDIA_DB_NAME = "tomatotodo-local-media";
  const LOCAL_MEDIA_DB_STORE = "handles";
  const LOCAL_MEDIA_HANDLE_KEY = "music-directory";
  const LOCAL_AUDIO_PATTERN = /\.(mp3|m4a|aac|wav|ogg|oga|flac|opus)$/i;
  const COUNTDOWN_STORAGE_KEY = "moke-countdown";
  const QUICK_NOTES_STORAGE_KEY = "moke-quick-notes";
  const QUICK_NOTE_GUIDE_VERSION_KEY =
    "tomatotodo-quick-note-guide-v7";
  const QUICK_NOTE_GUIDE_OPEN_KEY =
    "tomatotodo-quick-note-guide-open";
  const QUICK_NOTE_GUIDE_ID = "tomatotodo-app-guide";
  const QUOTE_MODE_STORAGE_KEY = "tomatotodo-quote-mode";
  const QUICK_NOTE_LAUNCHER_STORAGE_KEY =
    "tomatotodo-quick-note-launcher-visible";
  const DASHBOARD_GRID_STORAGE_KEY =
    "tomatotodo-dashboard-grid-v1";
  const IMMERSIVE_STORAGE_KEY = "tomatotodo-immersive-mode";
  const ANALOG_CLOCK_STORAGE_KEY =
    "tomatotodo-analog-clock-settings";
  const RAIL_COMPACT_STORAGE_KEY = "tomatotodo-rail-compact";
  const ACTIVE_TASK_STORAGE_KEY = "tomatotodo-active-task";
  const LOCAL_VOLUME_STORAGE_KEY = "tomatotodo-local-volume";
  const LOCAL_PLAYBACK_MODE_STORAGE_KEY =
    "tomatotodo-local-playback-mode";
  const USER_DATA_BACKUP_FORMAT = "tomatotodo-user-data";
  const USER_DATA_BACKUP_VERSION = 1;
  const FACTORY_DEFAULTS_STORAGE_KEY =
    "tomatotodo-factory-defaults-1.2";
  const FACTORY_DEFAULTS_VERSION = 1;
  const TASK_HISTORY_STORAGE_KEY = "tomatotodo-task-history-v1";
  const LIST_REPEAT_TYPES = new Set(["none", "daily", "weekdays",
    "weekly", "monthly", "yearly", "custom"]);
  const LIST_REPEAT_UNITS = new Set(["day", "week", "month",
    "year"]);
  const DASHBOARD_WIDGETS = {
    timer: { label: "大计时器", detail: "横版番茄钟显示", width:
      4, height: 2 },
    mode: { label: "专注 / 短休", detail: "切换计时阶段", width:
      2, height: 1 },
    countup: { label: "正向计时", detail: "碎片时间记录", width:
      1, height: 1 },
```

```

immersive: { label: "沉浸模式", detail: "进入全屏翻页时钟",
  width: 1, height: 1 },
miniwindow: { label: "小窗模式", detail: "显示系统置顶计时窗",
  width: 1, height: 1 },
tasks: { label: "任务清单", detail: "清单、任务与完成进度",
  width: 2, height: 2 },
quote: { label: "名言警句", detail: "专注与成长短句", width:
  2, height: 1 },
active: { label: "正在进行", detail: "当前分配任务", width: 2,
  height: 1 },
calendar: { label: "日历", detail: "本月日期", width: 2,
  height: 2 },
analogclock: { label: "圆表", detail: "简约时针、分针与秒针",
  width: 1, height: 2 },
media: { label: "当前媒体", detail: "Spotify 或本地音乐",
  width: 2, height: 1 },
date: { label: "日期与时间", detail: "今日日期和当前时间",
  width: 2, height: 1 },
weather: { label: "实时天气", detail: "当前位置天气", width:
  2, height: 1 },
countdown: { label: "倒数日", detail: "距离目标日期", width:
  2, height: 1 },
};
const DEFAULT_DASHBOARD_GRID_ORDER = [
  "timer", "tasks", "mode", "countup", "immersive",
  "miniwindow", "active", "calendar", "analogclock", "date",
  "weather", "countdown", "media", "quote",
];
function readDashboardGridLayout() {
  try {
    const saved = JSON.parse(localStorage.getItem(DASHBOARD_GRID
      _STORAGE_KEY));
    const known = new Set(Object.keys(DASHBOARD_WIDGETS));
    const savedOrder = Array.isArray(saved?.order) ?
      saved.order.filter((id) => known.has(id)) : [];
    const order = [...savedOrder,
      ...DEFAULT_DASHBOARD_GRID_ORDER.filter((id) =>
        !savedOrder.includes(id))];
    const visible = Object.fromEntries(order.map((id) => [id,
      saved?.visible?.[id] !== false]));
    return { order, visible };
  } catch {
    return {
      order: [...DEFAULT_DASHBOARD_GRID_ORDER],
      visible: Object.fromEntries(DEFAULT_DASHBOARD_GRID_ORDER.m
        ap((id) => [id, true])),
    };
  }
}
const TRANSLATOR_LAUNCHER_STORAGE_KEY =

```

```
"tomatotodo-translator-launcher-visible";
const CALCULATOR_LAUNCHER_STORAGE_KEY =
  "tomatotodo-calculator-launcher-visible";
const BING_TRANSLATOR_URL = "https://www.bing.com/translator?to=
  zh-Hans";
const FOCUS_QUOTES = [
  "日日行，不怕千万里；常常做，不怕千万事。",
  "把今天该做的事做好，明天自然会向你靠近。",
  "专注不是拒绝世界，而是把力量留给最重要的事。",
  "缓慢而坚定，比短暂的热烈更接近成长。",
  "不必一次抵达，持续前进本身就是答案。",
  "每一次沉下心，都是在为未来积蓄力量。",
  "把复杂留给过程，把简单留给当下这一刻。",
  "坚持不是重复昨天，而是让今天比昨天更进一步。",
  "真正的成长，常发生在无人喝彩的专注里。",
  "先完成眼前的一小步，再把远方交给时间。",
  "耐心是一种行动，它让微小的努力慢慢发光。",
  "保持专注，时间会把认真变成看得见的作品。",
];
const DAILY_FORTUNES = [
  { level: "大吉", quip:
    "今天的专注力正在偷偷超频，别告诉拖延症。"},
  { level: "中吉", quip:
    "适合先碰最难的任务——放心，只碰五分钟也算开始。"},
  { level: "小吉", quip:
    "进度会慢慢长出来，前提是别每两分钟挖开看看。"},
  { level: "吉", quip:
    "今日宜开始，忌打开任务后先整理桌面半小时。"},
  { level: "末吉", quip:
    "状态可能晚点到账，先开一个番茄钟替它占座。"},
  { level: "平", quip: "运气今天请假了，好在番茄钟还正常上班。"},
  },
  { level: "番茄吉", quip:
    "今天有番茄加成：认真二十五分钟，焦虑自动少一点。"},
];
const LOGO_PROCRASTINATION_MESSAGES = [
  { title: "不要再摸鱼了", body: "有这功夫，不如先学习五分钟。"},
  },
  { title: "Logo 不会掉落隐藏奖励", body:
    "但完成一个番茄钟，真的会留下进度。"},
  { title: "检测到高频点击", body:
    "手速很好，现在把它用在推进任务上吧。"},
  { title: "今日运势已经看过了", body:
    "真正能改运的，可能是现在开始。"},
  { title: "番茄正在看着你", body:
    "再点也不会成熟，专注二十五分钟试试。"},
  { title: "短暂休息可以", body: "但别把休息发展成 Logo
    压力测试。"},
  { title: "摸鱼额度接近上限", body:
    "建议立刻打开任务，保住今天的专注记录。"},
  },
];
```

```
{ title: "这不是抽卡按钮", body: "最佳奖励藏在完成任务之后。"
},
{ title: "运气加载完毕", body: "接下来该加载你的行动力了。"},
{ title: "点击次数很努力", body:
  "如果学习也保持这个频率，今天会很不错。"},
];
```

```
const QUICK_NOTE_GUIDE_CONTENT = `Tomatotodo 使用说明 | 1.3.0
```

Tomatotodo 是一款结合任务管理、番茄计时与专注记录的桌面应用。

一、快速开始

在“仪表盘”选择任务与专注时段，点击右上角播放按钮开始。计时可暂停或重置；启用短休后，专注与短休自动轮换。完成一轮专注会获得一枚番茄并写入档案。

正向计时适合碎片时间；沉浸模式提供全屏翻页时钟；小窗模式可在其他窗口上方显示当前状态。

二、任务与仪表盘

在“配置”中创建和管理任务清单。任务支持小标题、预计番茄数、完成进度与拖拽排序；清单可设置截止日期、提醒和重复规则。

仪表盘进入编辑状态后，可添加、关闭和移动组件；保存后会记住当前布局。

三、专注档案

“档案”展示今日数据、专注日历、单日志、统计折线图与年度热力图。右上角导出按钮可生成包含今日数据、日志、Logo 与水印的专注明信片。

四、工具与媒体

“工具”包含快捷便笺、翻译、计算器和秒表。便笺自动保存；点击便笺删除按钮并在确认弹窗中选择“删除”即可移除。

媒体组件支持 Spotify 与本地音乐。本地文件仅在当前设备读取，不会上传。

五、个性化与数据

“个性化”可调整自动、日间或夜间模式、Material You

主题色、纯黑模式和字体缩放。“常规”提供计时、圆表、倒数日、媒体及数据设置。

用户数据默认保存在本机。“常规”支持导入或导出 JSON

备份；Spotify 登录令牌、本地音乐文件与文件夹授权不会写入备份。恢复出厂数据需要两次确认。

六、快捷键

F12: 切换全屏。

F5: 刷新天气。

Esc: 关闭当前工具窗口。

更新日志 | 1.3.0

- 任务会保存已完成番茄进度，并以 2/4 等形式显示。
- 清单新增截止日期、提醒和重复计划，支持自定义日期、时间及星期。
- 新增历史任务，可批量恢复或删除已完成、已过期任务。
- 恢复任务时可选择目标清单，并决定是否保留番茄进度。
- 清单、任务与便笺删除统一使用简洁确认弹窗。
- 修复字体缩放低于 100% 时底部出现空白的问题。`;

```
function createQuickNoteGuide() {
  const now = new Date().toISOString();
  return {
```

```

    id: QUICK_NOTE_GUIDE_ID,
    content: QUICK_NOTE_GUIDE_CONTENT,
    createdAt: now,
    updatedAt: now,
    position: null,
  };
}
function readQuickNotes() {
  let notes = [];
  try {
    const saved = JSON.parse(localStorage.getItem(QUICK_NOTES_STORAGE_KEY));
    if (Array.isArray(saved)) {
      notes = saved
        .filter((note) => note && typeof note.id === "string")
        .map((note) => ({
          id: note.id,
          content: typeof note.content === "string" ?
            note.content : "",
          createdAt: note.createdAt || new Date().toISOString(),
          updatedAt: note.updatedAt || note.createdAt || new
            Date().toISOString(),
          position: note.position &&
            Number.isFinite(note.position.x) &&
            Number.isFinite(note.position.y)
            ? note.position
            : null,
        }));
    }
  } catch {
    notes = [];
  }
  try {
    if (localStorage.getItem(QUICK_NOTE_GUIDE_VERSION_KEY) !==
      "7") {
      const guide = createQuickNoteGuide();
      notes = [guide, ...notes.filter((note) => note.id !==
        QUICK_NOTE_GUIDE_ID)];
      localStorage.setItem(QUICK_NOTE_GUIDE_VERSION_KEY, "7");
      localStorage.setItem(QUICK_NOTES_STORAGE_KEY,
        JSON.stringify(notes));
      sessionStorage.setItem(QUICK_NOTE_GUIDE_OPEN_KEY, "1");
    }
  } catch {
    // Storage restrictions must not prevent the app from
    opening.
  }
  return notes;
}
function readInitialQuickNoteId() {

```

```
try {
  const shouldOpenGuide = sessionStorage.getItem(QUICK_NOTE_GUIDE_OPEN_KEY) === "1";
  sessionStorage.removeItem(QUICK_NOTE_GUIDE_OPEN_KEY);
  return shouldOpenGuide ? QUICK_NOTE_GUIDE_ID : null;
} catch {
  return null;
}
}

function quickNoteDate(value, includeTime = true) {
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return "--";
  return date.toLocaleString("zh-CN", includeTime
    ? { month: "2-digit", day: "2-digit", hour: "2-digit",
      minute: "2-digit", hour12: false }
    : { year: "numeric", month: "2-digit", day: "2-digit" });
}

function defaultQuickNotePosition() {
  if (typeof window === "undefined") return { x: 420, y: 110 };
  const cardWidth = Math.min(360, window.innerWidth - 32);
  return {
    x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
      window.innerWidth * 0.38)),
    y: Math.max(78, Math.min(window.innerHeight - 430,
      window.innerHeight * 0.14)),
  };
}

function clampQuickNotePosition(position) {
  if (typeof window === "undefined") return position;
  const cardWidth = Math.min(360, window.innerWidth - 32);
  const cardHeight = Math.min(420, window.innerHeight - 32);
  return {
    x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
      position.x)),
    y: Math.max(16, Math.min(window.innerHeight - cardHeight -
      16, position.y)),
  };
}

function defaultTranslatorPosition() {
  if (typeof window === "undefined") return { x: 430, y: 88 };
  const cardWidth = Math.min(560, window.innerWidth - 32);
  return {
    x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
      window.innerWidth * 0.34)),
    y: Math.max(72, Math.min(window.innerHeight - 520,
      window.innerHeight * 0.1)),
  };
}

function clampTranslatorPosition(position) {
  if (typeof window === "undefined") return position;
```



```
const cardWidth = Math.min(560, window.innerWidth - 32);
const cardHeight = Math.min(620, window.innerHeight - 32);
return {
  x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
    position.x)),
  y: Math.max(16, Math.min(window.innerHeight - cardHeight -
    16, position.y)),
};
}
function defaultCalculatorPosition() {
  if (typeof window === "undefined") return { x: 470, y: 110 };
  const cardWidth = Math.min(320, window.innerWidth - 32);
  return {
    x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
      window.innerWidth * 0.42)),
    y: Math.max(72, Math.min(window.innerHeight - 500,
      window.innerHeight * 0.13)),
  };
}
function clampCalculatorPosition(position) {
  if (typeof window === "undefined") return position;
  const cardWidth = Math.min(320, window.innerWidth - 32);
  const cardHeight = Math.min(500, window.innerHeight - 32);
  return {
    x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
      position.x)),
    y: Math.max(16, Math.min(window.innerHeight - cardHeight -
      16, position.y)),
  };
}
function defaultStopwatchPosition() {
  if (typeof window === "undefined") return { x: 500, y: 96 };
  const cardWidth = Math.min(360, window.innerWidth - 32);
  return {
    x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
      window.innerWidth * 0.46)),
    y: Math.max(72, Math.min(window.innerHeight - 520,
      window.innerHeight * 0.11)),
  };
}
function clampStopwatchPosition(position) {
  if (typeof window === "undefined") return position;
  const cardWidth = Math.min(360, window.innerWidth - 32);
  const cardHeight = Math.min(520, window.innerHeight - 32);
  return {
    x: Math.max(16, Math.min(window.innerWidth - cardWidth - 16,
      position.x)),
    y: Math.max(16, Math.min(window.innerHeight - cardHeight -
      16, position.y)),
  };
};
```

```

}
function formatStopwatchTime(milliseconds) {
  const totalCentiseconds = Math.max(0, Math.floor(milliseconds
    / 10));
  const minutes = Math.floor(totalCentiseconds / 6000);
  const seconds = Math.floor((totalCentiseconds % 6000) / 100);
  const centiseconds = totalCentiseconds % 100;
  return `${String(minutes).padStart(2, "0")}:${String(seconds)
    .padStart(2, "0")}:${String(centiseconds).padStart(2, "0")
    }`;
}
function readSpotifyAuth() {
  try {
    const saved = JSON.parse(localStorage.getItem(SPOTIFY_AUTH_S
      TORAGE_KEY));
    return saved?.accessToken && saved?.refreshToken ? saved :
      null;
  } catch {
    return null;
  }
}
function spotifyRedirectUri() {
  if (typeof window === "undefined") return "";
  const redirect = new URL(window.location.href);
  redirect.search = "";
  redirect.hash = "";
  return redirect.toString();
}
function spotifyLoopbackUrl(clientId = "") {
  if (typeof window === "undefined") return "";
  const url = new URL(window.location.href);
  url.hostname = "127.0.0.1";
  url.search = "";
  url.hash = clientId ? new URLSearchParams({
    spotify_client_id: clientId }).toString() : "";
  return url.toString();
}
function randomSpotifyString(length = 64) {
  const alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-._~";
  const values = crypto.getRandomValues(new Uint8Array(length));
  return Array.from(values, (value) => alphabet[value %
    alphabet.length]).join("");
}
async function spotifyCodeChallenge(verifier) {
  const digest = await crypto.subtle.digest("SHA-256", new
    TextEncoder().encode(verifier));
  return btoa(String.fromCharCode(...new Uint8Array(digest)))
    .replace(/=/g, "")
    .replace(/\+/g, "-")

```

```
.replace(/\\/g, "\\");
}
function storeSpotifyAuth(auth) {
  localStorage.setItem(SPOTIFY_AUTH_STORAGE_KEY,
    JSON.stringify(auth));
}
async function requestSpotifyToken(parameters) {
  const response = await fetch("https://accounts.spotify.com/api
    /token", {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: new URLSearchParams(parameters),
  });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(data.error_description ||
    data.error || "spotify-token-failed");
  return data;
}
async function buildSpotifyAuthorizationUrl(clientId) {
  const verifier = randomSpotifyString();
  const state = randomSpotifyString(32);
  const challenge = await spotifyCodeChallenge(verifier);
  localStorage.setItem(SPOTIFY_VERIFIER_STORAGE_KEY, verifier);
  localStorage.setItem(SPOTIFY_STATE_STORAGE_KEY, state);
  const authorizationUrl = new URL("https://accounts.spotify.com
    /authorize");
  authorizationUrl.search = new URLSearchParams({
    client_id: clientId,
    response_type: "code",
    redirect_uri: spotifyRedirectUri(),
    scope: "user-read-currently-playing
      user-read-playback-state",
    code_challenge_method: "S256",
    code_challenge: challenge,
    state,
  }).toString();
  return authorizationUrl.toString();
}
function readMediaSource() {
  try {
    return localStorage.getItem(MEDIA_SOURCE_STORAGE_KEY) ===
      "local" ? "local" : "spotify";
  } catch {
    return "spotify";
  }
}
function openLocalMediaDatabase() {
  return new Promise((resolve, reject) => {
    if (!window.indexedDB) {
```

```

    reject(new Error("indexeddb-unavailable"));
    return;
  }
  const request = window.indexedDB.open(LOCAL_MEDIA_DB_NAME,
    1);
  request.onupgradeneeded = () => {
    if (!request.result.objectStoreNames.contains(LOCAL_MEDIA_
      DB_STORE)) {
      request.result.createObjectStore(LOCAL_MEDIA_DB_STORE);
    }
  };
  request.onsuccess = () => resolve(request.result);
  request.onerror = () => reject(request.error);
});
}
async function storeLocalMediaHandle(handle) {
  const database = await openLocalMediaDatabase();
  await new Promise((resolve, reject) => {
    const transaction = database.transaction(LOCAL_MEDIA_DB_STORE,
      "readwrite");
    transaction.objectStore(LOCAL_MEDIA_DB_STORE).put(handle,
      LOCAL_MEDIA_HANDLE_KEY);
    transaction.oncomplete = resolve;
    transaction.onerror = () => reject(transaction.error);
  });
  database.close();
}
async function readLocalMediaHandle() {
  const database = await openLocalMediaDatabase();
  const handle = await new Promise((resolve, reject) => {
    const transaction = database.transaction(LOCAL_MEDIA_DB_STORE,
      "readonly");
    const request = transaction.objectStore(LOCAL_MEDIA_DB_STORE)
      .get(LOCAL_MEDIA_HANDLE_KEY);
    request.onsuccess = () => resolve(request.result || null);
    request.onerror = () => reject(request.error);
  });
  database.close();
  return handle;
}
async function collectLocalAudioFiles(directoryHandle, path =
  "") {
  const files = [];
  for await (const entry of directoryHandle.values()) {
    if (entry.kind === "file" &&
      LOCAL_AUDIO_PATTERN.test(entry.name)) {
      files.push({ file: await entry.getFile(), path });
    } else if (entry.kind === "directory") {
      files.push(...await collectLocalAudioFiles(entry, `${path}
        ${entry.name} / `));
    }
  }
}

```

```

    }
  }
  return files;
}

function decodeId3Text(bytes, encoding = 3) {
  if (!bytes.length) return "";
  try {
    if (encoding === 0) return new TextDecoder("iso-8859-1")
      .decode(bytes).replace(/\0/g, "").trim();
    if (encoding === 1) {
      const littleEndian = bytes[0] === 0xff && bytes[1] ===
        0xfe;
      const offset = (bytes[0] === 0xff && bytes[1] === 0xfe)
        || (bytes[0] === 0xfe && bytes[1] === 0xff) ? 2 : 0;
      return new TextDecoder(littleEndian ? "utf-16le" :
        "utf-16be").decode(bytes.slice(offset)).replace(/\0/g,
        "").trim();
    }
    if (encoding === 2) return new TextDecoder("utf-16be")
      .decode(bytes).replace(/\0/g, "").trim();
    return new TextDecoder("utf-8").decode(bytes).replace(/\0/g,
      "").trim();
  } catch {
    return "";
  }
}

function id3FrameSize(bytes, offset, version) {
  if (version === 4) return ((bytes[offset] & 0x7f) << 21) |
    ((bytes[offset + 1] & 0x7f) << 14) | ((bytes[offset + 2]
    & 0x7f) << 7) | (bytes[offset + 3] & 0x7f);
  return (bytes[offset] << 24) | (bytes[offset + 1] << 16) |
    (bytes[offset + 2] << 8) | bytes[offset + 3];
}

async function readLocalAudioMetadata(file, fallbackArtist =
  "本地音乐") {
  const fallbackTitle = file.name.replace(/\.([^.]*)$/, "");
  const metadata = { title: fallbackTitle, artist:
    fallbackArtist, album: "", artwork: "" };
  if (!/\.mp3$/i.test(file.name)) return metadata;
  try {
    const bytes = new Uint8Array(await file.slice(0,
      Math.min(file.size, 1024 * 1024)).arrayBuffer());
    if (String.fromCharCode(...bytes.slice(0, 3)) !== "ID3")
      return metadata;
    const version = bytes[3];
    const tagSize = ((bytes[6] & 0x7f) << 21) | ((bytes[7] &
      0x7f) << 14) | ((bytes[8] & 0x7f) << 7) | (bytes[9] &
      0x7f);
    let offset = 10;
    const limit = Math.min(bytes.length, 10 + tagSize);

```

```

while (offset + 10 <= limit) {
    const frameId = String.fromCharCode(...bytes.slice(offset,
        offset + 4));
    if (!/^[A-Z0-9]{4}$/.test(frameId)) break;
    const frameSize = id3FrameSize(bytes, offset + 4, version)
        ;
    if (frameSize <= 0 || offset + 10 + frameSize > limit)
        break;
    const frame = bytes.slice(offset + 10, offset + 10 +
        frameSize);
    if (frameId === "TIT2") metadata.title =
        decodeId3Text(frame.slice(1), frame[0]) ||
        metadata.title;
    if (frameId === "TPE1") metadata.artist =
        decodeId3Text(frame.slice(1), frame[0]) ||
        metadata.artist;
    if (frameId === "TALB") metadata.album =
        decodeId3Text(frame.slice(1), frame[0]);
    if (frameId === "APIC" && !metadata.artwork) {
        const encoding = frame[0];
        let cursor = 1;
        while (cursor < frame.length && frame[cursor] !== 0)
            cursor += 1;
        const mime = new TextDecoder("iso-8859-1")
            .decode(frame.slice(1, cursor)) || "image/jpeg";
        cursor += 2;
        const doubleTerminator = encoding === 1 || encoding ===
            2;
        while (cursor < frame.length - (doubleTerminator ? 1 :
            0)) {
            if (frame[cursor] === 0 && (!doubleTerminator ||
                frame[cursor + 1] === 0)) {
                cursor += doubleTerminator ? 2 : 1;
                break;
            }
            cursor += doubleTerminator ? 2 : 1;
        }
        if (cursor < frame.length) metadata.artwork =
            URL.createObjectURL(new Blob([frame.slice(cursor)],
                { type: mime }));
    }
    offset += 10 + frameSize;
}
} catch {
    // Filename and folder remain useful when metadata cannot
    be decoded.
}
return metadata;
}
function formatMediaTime(seconds) {

```

```
if (!Number.isFinite(seconds) || seconds < 0) return "0:00";
const whole = Math.floor(seconds);
return `${Math.floor(whole / 60)}:${String(whole % 60)
  .padStart(2, "0")}`;
}
async function copyTextToClipboard(text) {
  if (navigator.clipboard?.writeText && window.isSecureContext)
    {
      await navigator.clipboard.writeText(text);
      return;
    }
  const helper = document.createElement("textarea");
  helper.value = text;
  helper.setAttribute("readonly", "");
  helper.style.position = "fixed";
  helper.style.opacity = "0";
  document.body.appendChild(helper);
  helper.select();
  const copied = document.execCommand("copy");
  helper.remove();
  if (!copied) throw new Error("copy-failed");
}
const INITIAL_TASKS = [
  { id: 1, title: "素描头像结构练习", done: false,
    estimatedPomodoros: null, completedPomodoros: 0 },
  { id: 2, title: "色彩静物小稿", done: false,
    estimatedPomodoros: null, completedPomodoros: 0 },
  { id: 3, title: "速写人物动态", done: false,
    estimatedPomodoros: null, completedPomodoros: 0 },
];
const MONET_PRESETS = [
  { id: "forest", name: "青瓷", seed: "#30643B" },
  { id: "coral", name: "珊瑚", seed: "#9B4938" },
  { id: "sky", name: "晴空", seed: "#426785" },
  { id: "olive", name: "橄榄", seed: "#6B6918" },
  { id: "cyan", name: "湖蓝", seed: "#216775" },
  { id: "meadow", name: "草甸", seed: "#4E6B34" },
  { id: "blossom", name: "花信", seed: "#8C5265" },
  { id: "violet", name: "紫藤", seed: "#6F5A82" },
];
const MATERIAL_SCHEMES = [
  { id: "tonalSpot", label: "调性点缀", Scheme: SchemeTonalSpot
  },
  { id: "fidelity", label: "高保真", Scheme: SchemeFidelity },
  { id: "monochrome", label: "单色", Scheme: SchemeMonochrome },
  { id: "neutral", label: "中性", Scheme: SchemeNeutral },
  { id: "vibrant", label: "活力", Scheme: SchemeVibrant },
  { id: "expressive", label: "表现力", Scheme: SchemeExpressive
  },
  { id: "content", label: "内容主题", Scheme: SchemeContent },
```

```

    { id: "rainbow", label: "彩虹", Scheme: SchemeRainbow },
    { id: "fruitSalad", label: "果缤纷", Scheme: SchemeFruitSalad
    },
];

function darkenHex(hex, amount = 0.24) {
    const value = hex.replace("#", "");
    if (!/^[0-9a-f]{6}$/i.test(value)) return "#234d2d";
    const channels = [0, 2, 4].map((index) =>
        Math.max(0, Math.round(parseInt(value.slice(index, index +
            2), 16) * (1 - amount))),
    );
    return `#${channels.map((channel) => channel.toString(16)
        .padStart(2, "0")).join("")}`;
}

function hexToRgb(hex) {
    const value = hex.replace("#", "");
    if (!/^[0-9a-f]{6}$/i.test(value)) return { r: 48, g: 100, b:
        59 };
    return {
        r: parseInt(value.slice(0, 2), 16),
        g: parseInt(value.slice(2, 4), 16),
        b: parseInt(value.slice(4, 6), 16),
    };
}

function rgbToHex({ r, g, b }) {
    return `#${[r, g, b]
        .map((channel) => Math.round(channel).toString(16)
            .padStart(2, "0"))
        .join("")}`;
}

function mixHex(base, tint, tintAmount) {
    const a = hexToRgb(base);
    const b = hexToRgb(tint);
    return rgbToHex({
        r: a.r * (1 - tintAmount) + b.r * tintAmount,
        g: a.g * (1 - tintAmount) + b.g * tintAmount,
        b: a.b * (1 - tintAmount) + b.b * tintAmount,
    });
}

function rgbaFromHex(hex, alpha) {
    const { r, g, b } = hexToRgb(hex);
    return `rgba(${r}, ${g}, ${b}, ${alpha})`;
}

function relativeLuminance(hex) {
    const { r, g, b } = hexToRgb(hex);
    const linear = [r, g, b].map((channel) => {
        const value = channel / 255;
        return value <= 0.03928 ? value / 12.92 : ((value + 0.055)
            / 1.055) ** 2.4;
    });
};

```



```
    return linear[0] * 0.2126 + linear[1] * 0.7152 + linear[2] *
      0.0722;
  }
  function createMaterialScheme(seed, appearance, variantId =
    "tonalSpot") {
    const Scheme = MATERIAL_SCHEMES.find((item) => item.id ===
      variantId)?.Scheme || SchemeTonalSpot;
    return new Scheme(Hct.fromInt(ArgbFromHex(seed)), appearance
      === "dark", 0);
  }
  function materialHex(value) {
    return hexFromArgb(value).toUpperCase();
  }
  function materialPreviewColors(seed, variantId = "tonalSpot") {
    const scheme = createMaterialScheme(seed, "dark", variantId);
    return [
      materialHex(scheme.primary),
      materialHex(scheme.secondaryContainer),
      materialHex(scheme.tertiaryContainer),
      materialHex(scheme.surfaceContainerHighest),
    ];
  }
  function buildMonetTokens(seed, appearance, variantId =
    "tonalSpot", pureBlack = false) {
    const isLight = appearance === "light";
    const scheme = createMaterialScheme(seed, appearance,
      variantId);
    const primary = materialHex(scheme.primary);
    const primaryContainer = materialHex(scheme.primaryContainer);
    const surface = materialHex(scheme.surface);
    const surfaceContainer = materialHex(scheme.surfaceContainer);
    const surfaceRaised = materialHex(scheme.surfaceContainerHigh)
      ;
    const text = materialHex(scheme.onSurface);
    const muted = materialHex(scheme.onSurfaceVariant);
    const softText = materialHex(scheme.outline);
    const modalBase = materialHex(scheme.surfaceContainerLow);
    const modalPanel = materialHex(scheme.surfaceContainerHigh);
    const usePureBlack = pureBlack && !isLight;
    return {
      "--accent": primary,
      "--accent-deep": primaryContainer,
      "--bg": usePureBlack ? "#000000" : surface,
      "--surface": usePureBlack ? mixHex("#050505", primary,
        0.055) : surfaceContainer,
      "--surface-raised": usePureBlack ? mixHex("#0B0B0B",
        primary, 0.075) : surfaceRaised,
      "--text": text,
      "--muted": muted,
      "--soft-text": softText,
```

```

"--line": rgbaFromHex(materialHex(scheme.outlineVariant),
  isLight ? 0.66 : 0.74),
"--line-strong": rgbaFromHex(materialHex(scheme.outline),
  isLight ? 0.78 : 0.82),
"--line-soft": rgbaFromHex(materialHex(scheme.outlineVariant
), 0.38),
"--track": materialHex(scheme.surfaceContainerHighest),
"--switch-bg": materialHex(scheme.surfaceVariant),
"--switch-border": materialHex(scheme.outline),
"--shadow": isLight ? rgbaFromHex(materialHex(scheme.shadow
, 0.18) : rgbaFromHex(materialHex(scheme.shadow), 0.44),
"--on-accent": materialHex(scheme.onPrimary),
"--modal-bg": rgbaFromHex(usePureBlack ? "#050505" :
  modalBase, isLight ? 0.92 : 0.94),
"--modal-panel": rgbaFromHex(usePureBlack ? "#0A0A0A" :
  modalPanel, isLight ? 0.78 : 0.76),
"--modal-panel-strong": rgbaFromHex(usePureBlack ?
  "#101010" : materialHex(scheme.surfaceContainerHighest),
  isLight ? 0.9 : 0.86),
"--modal-line": rgbaFromHex(materialHex(scheme.outlineVari
t), isLight ? 0.58 : 0.68),
"--modal-shadow": isLight ? rgbaFromHex(materialHex(scheme.s
hadow), 0.26) : "rgba(0, 0, 0, 0.58)",
};
}
function describeWeatherCode(code) {
  if (code === 0) return "晴";
  if ([1, 2].includes(code)) return "晴间多云";
  if (code === 3) return "阴";
  if ([45, 48].includes(code)) return "有雾";
  if ([51, 53, 55, 56, 57].includes(code)) return "毛毛雨";
  if ([61, 63, 65, 66, 67].includes(code)) return "有雨";
  if ([71, 73, 75, 77, 85, 86].includes(code)) return "有雪";
  if ([80, 81, 82].includes(code)) return "阵雨";
  if ([95, 96, 99].includes(code)) return "雷雨";
  return "天气变化中";
}
function parseTaskEstimate(value) {
  if (value === "" || value == null) return null;
  const parsed = Number.parseInt(value, 10);
  return Number.isFinite(parsed) ? Math.min(99, Math.max(1,
    parsed)) : null;
}
function clampTaskEstimateInput(value) {
  if (value === "") return "";
  const parsed = Number.parseInt(value, 10);
  if (!Number.isFinite(parsed)) return "";
  return String(Math.min(99, Math.max(1, parsed)));
}
function parseTaskPomodoroProgress(value, estimatedPomodoros =

```

```

    null) {
    const parsed = Number.parseInt(value, 10);
    const progress = Number.isFinite(parsed) ? Math.max(0, parsed)
      : 0;
    const estimate = parseTaskEstimate(estimatedPomodoros);
    return estimate ? Math.min(progress, estimate) : progress;
  }
function completedPomodoroCountsFromLogs(logs) {
  return (Array.isArray(logs) ? logs : []).reduce((counts, log)
    => {
    if (log?.reason !== "completed" || log.taskId == null)
      return counts;
    counts.set(log.taskId, (counts.get(log.taskId) || 0) + 1);
    return counts;
  }, new Map());
}
function normalizeTasks(tasks, completedCounts = null) {
  if (!Array.isArray(tasks)) return [];
  return tasks.map((task) => {
    const estimatedPomodoros = parseTaskEstimate(task?.estimated
      Pomodoros);
    const savedProgress = parseTaskPomodoroProgress(task?.comple
      tedPomodoros, estimatedPomodoros);
    const legacyProgress = completedCounts?.get(task?.id) || 0;
    return {
      ...task,
      estimatedPomodoros,
      completedPomodoros: parseTaskPomodoroProgress(Math.max(sav
        edProgress, legacyProgress), estimatedPomodoros),
    };
  });
}
function normalizeListRepeat(repeat) {
  const source = typeof repeat === "string" ? { type: repeat }
    : repeat;
  const type = LIST_REPEAT_TYPES.has(source?.type) ?
    source.type : "none";
  const interval = Math.min(99, Math.max(1,
    Number.parseInt(source?.interval, 10) || 1));
  const unit = LIST_REPEAT_UNITS.has(source?.unit) ?
    source.unit : "week";
  const weekdays = Array.isArray(source?.weekdays)
    ? [...new Set(source.weekdays.map((day) =>
      Number.parseInt(day, 10)).filter((day) => day >= 0 &&
        day <= 6))].sort((a, b) => a - b)
    : [];
  return { type, interval, unit, weekdays };
}
function normalizeTaskList(list, completedCounts = null) {
  const dueDate = /^d{4}-d{2}-d{2}$/ .test(list?.dueDate ||

```

```

    "" ? list.dueDate : "";
const reminderAt = Number.isFinite(Date.parse(list?.reminderAt
  || "")) ? list.reminderAt : "";
const reminderFiredAt = list?.reminderFiredAt === reminderAt
  ? reminderAt : "";
return {
  ...list,
  dueDate,
  reminderAt,
  reminderFiredAt,
  repeat: normalizeListRepeat(list?.repeat),
  tasks: normalizeTasks(list?.tasks, completedCounts),
};
}
function normalizeTaskHistoryEntry(entry) {
  if (!entry || !entry.task || typeof entry.task.title !==
    "string") return null;
  const status = entry.status === "expired" ? "expired" :
    "completed";
  return {
    id: String(entry.id || `history-${Date.now()}
      -${Math.random().toString(36).slice(2, 8)}`),
    status,
    archivedAt: Number.isFinite(Date.parse(entry.archivedAt ||
      "")) ? entry.archivedAt : new Date().toISOString(),
    sourceListId: String(entry.sourceListId || ""),
    sourceListName: String(entry.sourceListName || "未命名清单"),
    ,
    dueDate: /^\\d{4}-\\d{2}-\\d{2}$/.test(entry.dueDate || "") ?
      entry.dueDate : "",
    task: normalizeTasks([...entry.task, done: true])[0],
  };
}
function readTaskHistory() {
  try {
    const saved = JSON.parse(localStorage.getItem(TASK_HISTORY_S
      TORAGE_KEY));
    return Array.isArray(saved) ?
      saved.map(normalizeTaskHistoryEntry).filter(Boolean) :
      [];
  } catch {
    return [];
  }
}
function createTaskHistoryEntry(task, list, status) {
  return normalizeTaskHistoryEntry({
    id: `history-${Date.now()}-${task.id}-${Math.random()
      .toString(36).slice(2, 7)}`,
    status,
    archivedAt: new Date().toISOString(),
  });
}

```

```

        sourceListId: list?.id || "",
        sourceListName: list?.name || "未命名清单",
        dueDate: list?.dueDate || "",
        task: { ...task, done: true },
    });
}
function readStoredTasks() {
    const completedCounts = completedPomodoroCountsFromLogs(readFocusLogs());
    try {
        const saved = JSON.parse(localStorage.getItem("moke-tasks"));
        ;
        return Array.isArray(saved) ? normalizeTasks(saved,
            completedCounts) : normalizeTasks(INITIAL_TASKS,
            completedCounts);
    } catch {
        return normalizeTasks(INITIAL_TASKS, completedCounts);
    }
}
function readTodayStats() {
    const today = new Date().toLocaleDateString("zh-CN");
    try {
        const saved = JSON.parse(localStorage.getItem("moke-stats"));
        ;
        return saved?.date === today ? saved : { date: today,
            sessions: 0, minutes: 0 };
    } catch {
        return { date: today, sessions: 0, minutes: 0 };
    }
}
function readAppearance(key, fallback) {
    try {
        return localStorage.getItem(key) || fallback;
    } catch {
        return fallback;
    }
}
function readActiveTaskId() {
    try {
        const saved = JSON.parse(localStorage.getItem(ACTIVE_TASK_STORAGE_KEY));
        return saved ?? readStoredTasks()[0]?.id ?? null;
    } catch {
        return readStoredTasks()[0]?.id ?? null;
    }
}
function readInitialWorkspaceView() {
    if (typeof window === "undefined") return "dashboard";
    const view = new URLSearchParams(window.location.search)
        .get("view");

```

```
    return ["config", "personalization", "general", "archive",
        "tools"].includes(view) ? view : "dashboard";
}
function readQuoteMode() {
    return readAppearance(QUOTE_MODE_STORAGE_KEY, "false") ===
        "true";
}
function readQuickNoteLauncherVisible() {
    return readAppearance(QUICK_NOTE_LAUNCHER_STORAGE_KEY, "true")
        !== "false";
}
function readTranslatorLauncherVisible() {
    return readAppearance(TRANSLATOR_LAUNCHER_STORAGE_KEY,
        "false") === "true";
}
function readCalculatorLauncherVisible() {
    return readAppearance(CALCULATOR_LAUNCHER_STORAGE_KEY,
        "false") === "true";
}
function runBasicCalculation(left, operator, right) {
    if (operator === "+") return left + right;
    if (operator === "-") return left - right;
    if (operator === "x") return left * right;
    if (operator === "÷") return right === 0 ? NaN : left /
        right;
    return right;
}
function formatCalculatorValue(value) {
    if (!Number.isFinite(value)) return "错误";
    const magnitude = Math.abs(value);
    if ((magnitude >= 1e12) || (magnitude > 0 && magnitude < 1e-8)
        ) return value.toExponential(7).replace("e+", "e");
    return Number.parseFloat(value.toPrecision(12)).toString();
}
function readContextLayout() {
    const fallback = {
        order: ["date", "weather", "countdown", "calendar", "media"]
        ,
        visible: { date: true, weather: true, countdown: true,
            calendar: true, media: true },
    };
    try {
        const saved = JSON.parse(localStorage.getItem("moke-context-
            layout"));
        const savedOrder = Array.isArray(saved?.order)
            ? saved.order.filter((item) =>
                Object.hasOwn(CONTEXT_COMPONENTS, item))
            : [];
        const order = savedOrder.length ? [...new Set(savedOrder)]
            : [...fallback.order];
    }
```

```

    fallback.order.forEach((componentId) => {
      if (order.includes(componentId)) return;
      if (componentId === "countdown") {
        const weatherIndex = order.indexOf("weather");
        order.splice(weatherIndex >= 0 ? weatherIndex + 1 :
          order.length, 0, componentId);
        return;
      }
      order.push(componentId);
    });
    return {
      order,
      visible: {
        date: saved?.visible?.date !== false,
        weather: saved?.visible?.weather !== false,
        countdown: saved?.visible?.countdown !== false,
        calendar: saved?.visible?.calendar !== false,
        media: saved?.visible?.media !== false,
      },
    };
  } catch {
    return fallback;
  }
}

function localDateInputValue(date) {
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");
  return `${year}-${month}-${day}`;
}

function localDateTimeInputValue(date) {
  return `${localDateInputValue(date)}T${String(date.getHours())
    .padStart(2, "0")}:${String(date.getMinutes()).padStart(2,
    "0")}`;
}

function nextWeekDate(source = new Date()) {
  const date = new Date(source);
  date.setHours(12, 0, 0, 0);
  date.setDate(date.getDate() + 7);
  return date;
}

function laterTodayDate(source = new Date()) {
  const date = new Date(source);
  const preferredHour = date.getHours() < 15 ? 15 :
    date.getHours() < 18 ? 18 : 21;
  date.setHours(preferredHour, 0, 0, 0);
  if (date <= source) date.setTime(source.getTime() + 15 * 60 *
    1000);
  return date;
}

```

```
function formatChineseWeekday(date, full = true) {
  const labels = full
    ? ["星期日", "星期一", "星期二", "星期三", "星期四",
      "星期五", "星期六"]
    : ["周日", "周一", "周二", "周三", "周四", "周五", "周六"];
  return labels[date.getDay()];
}

function formatChineseClock(date, includeWeekday = false) {
  const hour = date.getHours();
  const minute = String(date.getMinutes()).padStart(2, "0");
  const period = hour < 12 ? "上午" : "下午";
  const displayHour = hour % 12 || 12;
  return `${includeWeekday ? `${formatChineseWeekday(date,
    false)} ` : ""}${period}${displayHour}:${minute}`;
}

function addListRepeatInterval(source, repeat) {
  const rule = normalizeListRepeat(repeat);
  const date = new Date(source);
  if (!Number.isFinite(date.getTime()) || rule.type === "none")
    return date;
  const addMonthsClamped = (monthCount) => {
    const originalDay = date.getDate();
    date.setDate(1);
    date.setMonth(date.getMonth() + monthCount);
    const lastDay = new Date(date.getFullYear(), date.getMonth()
      + 1, 0).getDate();
    date.setDate(Math.min(originalDay, lastDay));
  };
  const addYearsClamped = (yearCount) => {
    const originalMonth = date.getMonth();
    const originalDay = date.getDate();
    date.setDate(1);
    date.setFullYear(date.getFullYear() + yearCount);
    date.setMonth(originalMonth);
    const lastDay = new Date(date.getFullYear(), originalMonth
      + 1, 0).getDate();
    date.setDate(Math.min(originalDay, lastDay));
  };
  if (rule.type === "daily") date.setDate(date.getDate() + 1);
  if (rule.type === "weekdays") {
    const day = date.getDay();
    date.setDate(date.getDate() + (day === 5 ? 3 : day === 6 ?
      2 : 1));
  }
  if (rule.type === "weekly") date.setDate(date.getDate() + 7);
  if (rule.type === "monthly") addMonthsClamped(1);
  if (rule.type === "yearly") addYearsClamped(1);
  if (rule.type === "custom") {
    if (rule.unit === "day") date.setDate(date.getDate() +
      rule.interval);
  }
}
```



```

    if (rule.unit === "week") {
      if (!rule.weekdays.length) {
        date.setDate(date.getDate() + rule.interval * 7);
      } else {
        const currentDay = date.getDay();
        const nextDayThisCycle = rule.weekdays.find((day) =>
          day > currentDay);
        const daysToAdd = nextDayThisCycle != null
          ? nextDayThisCycle - currentDay
          : (7 - currentDay + rule.weekdays[0]) +
            ((rule.interval - 1) * 7);
        date.setDate(date.getDate() + Math.max(1, daysToAdd));
      }
    }
    if (rule.unit === "month") addMonthsClamped(rule.interval);
    if (rule.unit === "year") addYearsClamped(rule.interval);
  }
  return date;
}

function formatListDueDate(dateValue) {
  if (!dateValue) return "未设置";
  const today = new Date();
  const tomorrow = new Date(today);
  tomorrow.setDate(today.getDate() + 1);
  if (dateValue === localDateInputValue(today)) return "今天";
  if (dateValue === localDateInputValue(tomorrow)) return
    "明天";
  const date = new Date(`${dateValue}T12:00:00`);
  return Number.isFinite(date.getTime()) ? `${date.getMonth() +
    1}月${date.getDate()}日` : "未设置";
}

function formatListReminder(dateValue) {
  if (!dateValue) return "不提醒";
  const date = new Date(dateValue);
  if (!Number.isFinite(date.getTime())) return "不提醒";
  const dateLabel = formatListDueDate(localDateInputValue(date))
    ;
  return `${dateLabel} ${String(date.getHours()).padStart(2,
    "0")}:${String(date.getMinutes()).padStart(2, "0")}`;
}

function formatListRepeat(repeat) {
  const rule = normalizeListRepeat(repeat);
  const labels = {
    none: "不重复",
    daily: "每一天",
    weekdays: "工作日",
    weekly: "每一周",
    monthly: "每个月",
    yearly: "每一年",
  };
};

```

```

    if (rule.type !== "custom") return labels[rule.type] ||
      labels.none;
    const units = { day: "天", week: "周", month: "个月", year:
      "年" };
    const weekdayLabels = ["周日", "周一", "周二", "周三", "周四",
      "周五", "周六"];
    const weekdayCopy = rule.unit === "week" &&
      rule.weekdays.length
      ? ` ${rule.weekdays.map((day) => weekdayLabels[day])
        .join("、")}`
      : "";
    return `每 ${rule.interval} ${units[rule.unit]}${weekdayCopy}
    `;
  }
  function dailyFortuneFor(date) {
    const daySeed = date.getFullYear() * 372 + (date.getMonth() +
      1) * 31 + date.getDate();
    return DAILY_FORTUNES[Math.abs(daySeed * 17 + 11) %
      DAILY_FORTUNES.length];
  }
  function blobToDataURL(blob) {
    return new Promise((resolve, reject) => {
      const reader = new FileReader();
      reader.onload = () => resolve(reader.result);
      reader.onerror = () => reject(reader.error || new
        Error("图片读取失败。"));
      reader.readAsDataURL(blob);
    });
  }
  async function requestPngSaveHandle(suggestedName) {
    if (typeof window.showSaveFilePicker !== "function") return
      null;
    try {
      return await window.showSaveFilePicker({
        suggestedName,
        types: [{ description: "PNG 图片", accept: { "image/png":
          [".png"] } }],
        excludeAcceptAllOption: true,
      });
    } catch (error) {
      if (error?.name === "AbortError") return false;
      return null;
    }
  }
  async function savePngBlob(blob, fileName, saveHandle) {
    if (saveHandle) {
      const writable = await saveHandle.createWritable();
      try {
        await writable.write(blob);
      } finally {

```

```
        await writable.close();
    }
    return "picker";
}
const url = URL.createObjectURL(blob);
try {
    const anchor = document.createElement("a");
    anchor.href = url;
    anchor.download = fileName;
    anchor.rel = "noopener";
    anchor.style.display = "none";
    document.body.appendChild(anchor);
    anchor.click();
    anchor.remove();
    await new Promise((resolve) => window.setTimeout(resolve,
        250));
} finally {
    URL.revokeObjectURL(url);
}
return "download";
}
function focusArchiveCareMessage(date) {
    const hour = date.getHours();
    if (hour < 5) return
        "夜深了，先照顾好自己；未完成的事，也可以留给天亮以后。";
    if (hour < 8) return
        "早晨很安静，慢慢开始，让第一段专注为今天定下节奏。";
    if (hour < 12) return
        "上午思路正清晰，稳稳推进眼前最重要的一件事。";
    if (hour < 14) return
        "中午了，记得好好吃饭和稍作休息，让专注也有余地。";
    if (hour < 18) return
        "午后容易疲惫，放慢一点也没关系，你仍在认真向前。";
    if (hour < 20) return
        "傍晚了，收好今天的进展，也给自己留一点喘息。";
    return "晚上适合沉下心，但别忘了为睡眠和明天保留能量。";
}
function defaultCountdown() {
    const target = new Date();
    target.setHours(12, 0, 0, 0);
    target.setDate(target.getDate() + 30);
    return { label: "目标日", date: localDateInputValue(target) };
}
function readCountdown() {
    const fallback = defaultCountdown();
    try {
        const saved = JSON.parse(localStorage.getItem(COUNTDOWN_STOR
            AGE_KEY));
        const label = typeof saved?.label === "string" &&
            saved.label.trim()
```

```
    ? saved.label.trim().slice(0, 24)
    : fallback.label;
const date = /^\\d{4}-\\d{2}-\\d{2}$/.test(saved?.date || "")
    ? saved.date : fallback.date;
return { label, date };
} catch {
    return fallback;
}
}

function countdownDays(targetDate, now) {
    const [year, month, day] = targetDate.split("-").map(Number);
    const targetUtc = Date.UTC(year, month - 1, day);
    const todayUtc = Date.UTC(now.getFullYear(), now.getMonth(),
        now.getDate());
    return Math.max(0, Math.ceil((targetUtc - todayUtc) /
        86400000));
}

function readThemeMode() {
    const saved = readAppearance("moke-theme-mode",
        readAppearance("moke-theme", "dark"));
    return ["light", "dark", "auto"].includes(saved) ? saved :
        "dark";
}

function readThemeVariant() {
    const saved = readAppearance("moke-theme-variant",
        "tonalSpot");
    return MATERIAL_SCHEMES.some((item) => item.id === saved) ?
        saved : "tonalSpot";
}

function readFontScale() {
    const saved = Number.parseInt(readAppearance("moke-font-scale"
        , "100"), 10);
    return Number.isFinite(saved) ? Math.min(125, Math.max(85,
        saved)) : 100;
}

function resolveAutomaticTheme(now, sunTimes) {
    if (sunTimes?.sunrise && sunTimes?.sunset) {
        return now >= sunTimes.sunrise && now < sunTimes.sunset ?
            "light" : "dark";
    }
    const hour = now.getHours();
    return hour >= 6 && hour < 18 ? "light" : "dark";
}

function readTimerSettings() {
    const fallback = {
        focus: 25,
        short: 5,
        shortBreakEnabled: true,
        countUpEnabled: false,
    };
};
```

```

    try {
      const saved = JSON.parse(localStorage.getItem("moke-timer-se
        ttings"));
      return saved ? { ...fallback, ...saved } : fallback;
    } catch {
      return fallback;
    }
  }
}

function readAnalogClockSettings() {
  const fallback = { showMarkers: true, showSeconds: true };
  try {
    const saved = JSON.parse(localStorage.getItem(ANALOG_CLOCK_S
      TORAGE_KEY));
    return saved ? {
      showMarkers: saved.showMarkers !== false,
      showSeconds: saved.showSeconds !== false,
    } : fallback;
  } catch {
    return fallback;
  }
}

function readFocusLogs() {
  try {
    return JSON.parse(localStorage.getItem("moke-focus-logs"))
      || [];
  } catch {
    return [];
  }
}

function readTaskLists() {
  const completedCounts = completedPomodoroCountsFromLogs(readFo
    cusLogs());
  try {
    const saved = JSON.parse(localStorage.getItem("moke-task-lis
      ts"));
    if (Array.isArray(saved) && saved.length) {
      return saved.map((list) => normalizeTaskList({
        ...list,
        name: list.id === "today" && list.name === "今日清单" ?
          "今日任务" : list.name,
      }, completedCounts));
    }
  } catch {
    // Fall through to the first local list.
  }
  return [
    normalizeTaskList({ id: "today", name: "今日任务", tasks:
      readStoredTasks() }, completedCounts),
    normalizeTaskList({
      id: "foundation",

```

```

name: "央美基础训练",
tasks: [
  { id: 101, title: "素描头像结构练习", done: false,
    estimatedPomodoros: null, completedPomodoros: 0 },
  { id: 102, title: "色彩静物小稿", done: false,
    estimatedPomodoros: null, completedPomodoros: 0 },
  { id: 103, title: "速写人物动态", done: false,
    estimatedPomodoros: null, completedPomodoros: 0 },
],
}, completedCounts),
];
}
function toDateKey(date) {
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");
  return `${year}-${month}-${day}`;
}
function monthCells(date) {
  const first = new Date(date.getFullYear(), date.getMonth(), 1)
    ;
  const offset = (first.getDay() + 6) % 7;
  const start = new Date(date.getFullYear(), date.getMonth(), 1
    - offset);
  return Array.from({ length: 42 }, (_, index) => {
    const day = new Date(start);
    day.setDate(start.getDate() + index);
    return day;
  });
}
function formatMinutes(seconds) {
  if (!seconds) return "0 分钟";
  if (seconds < 60) return `${seconds} 秒`;
  const minutes = Math.floor(seconds / 60);
  const remainder = seconds % 60;
  return remainder ? `${minutes} 分 ${remainder} 秒` :
    `${minutes} 分钟`;
}
function startOfWeek(date) {
  const start = new Date(date.getFullYear(), date.getMonth(),
    date.getDate());
  start.setDate(start.getDate() - ((start.getDay() + 6) % 7));
  return start;
}
function focusSeries(logs, granularity, cursor) {
  const secondsByKey = logs.reduce((map, log) => {
    map.set(log.date, (map.get(log.date) || 0) + log.seconds);
    return map;
  }, new Map());
  const minutes = (seconds) => Math.round((seconds / 60) * 10)

```

```

        radius);
    if (opacity <= 0) continue;
    const dot = Math.abs((x / distance) * lightX + (-y /
        distance) * lightY);
    const edge = Math.sqrt(Math.max(0, 1 - (1 - fromEdge) **
        2));
    const coefficient = dot * edge;
    const color = Math.max(0, Math.min(255, (255 *
        coefficient) | 0));
    const alpha = Math.max(0, Math.min(255, (color *
        coefficient * opacity) | 0));
    const pixelIndex = (yPosition * width + xPosition) * 4;
    pixels[pixelIndex] = color;
    pixels[pixelIndex + 1] = color;
    pixels[pixelIndex + 2] = color;
    pixels[pixelIndex + 3] = alpha;
  }
}
context.putImageData(image, 0, 0);
return canvas.toDataURL();
}
export default function LiquidGlassPane({ children,
  borderRadius = 30, className = "" }) {
  const paneRef = useRef(null);
  const blurReplicaRef = useRef(null);
  const replicaRef = useRef(null);
  const filterId = `liquid-glass-${useId().replace(/:/g, "-")}`;
  const [filter, setFilter] = useState(null);
  useEffect(() => {
    const pane = paneRef.current;
    if (!pane) return undefined;
    let frame = 0;
    const rebuild = () => {
      window.cancelAnimationFrame(frame);
      frame = window.requestAnimationFrame(() => {
        const rect = pane.getBoundingClientRect();
        const width = Math.max(2, Math.round(rect.width));
        const height = Math.max(2, Math.round(rect.height));
        const radius = Math.min(borderRadius, width / 2 - 1,
          height / 2 - 1);
        const bezel = Math.max(8, Math.min(26, radius - 1));
        const cacheKey = `${width}x${height}-${radius}-${bezel}
          `;
        const cached = filterCache.get(cacheKey);
        if (cached) {
          setFilter(cached);
          return;
        }
      });
    }
    // React Bits FluidGlass uses a low IOR (1.15) and a
    thick, clear medium.
  }, [borderRadius, width, height, radius, bezel]);

```

```

// Keep the same optical character while using a
// DOM-safe displacement map.
const profile = calculateRefractionProfile(10, bezel,
  1.15);
const maxDisplacement = Math.max(...Array.from(profile)
  .map(Math.abs)) || 1;
const nextFilter = {
  width,
  height,
  // Keep the flat centre optically calm, but make the
  // rounded edge
  // visibly pull and bend the page beneath it.
  scale: Math.max(74, Math.min(104, bezel * 3.7)),
  displacement: generateDisplacementMap(width, height,
    radius, bezel, profile, maxDisplacement),
  specular: generateSpecularMap(width, height, radius,
    bezel * 2.35),
};
if (filterCache.size >= 8)
  filterCache.delete(filterCache.keys().next().value);
filterCache.set(cacheKey, nextFilter);
setFilter(nextFilter);
});
};
rebuild();
const observer = new ResizeObserver(rebuild);
observer.observe(pane);
return () => {
  observer.disconnect();
  window.cancelAnimationFrame(frame);
};
}, [borderRadius]);
useEffect(() => {
  const pane = paneRef.current;
  const blurReplica = blurReplicaRef.current;
  const replica = replicaRef.current;
  const source = document.querySelector(".app-shell");
  if (!pane || !blurReplica || !replica || !source) return
    undefined;
  const prepareClone = () => {
    const nextClone = source.cloneNode(true);
    nextClone.setAttribute("aria-hidden", "true");
    nextClone.querySelectorAll(".quick-note-shell,
      .quick-note-launcher, .quick-note-history")
      .forEach((node) => node.remove());
    nextClone.querySelectorAll("[id]").forEach((node) =>
      node.removeAttribute("id"));
    nextClone.querySelectorAll("button, input, textarea,
      select, a").forEach((node) =>
      node.setAttribute("tabindex", "-1"));
  };

```



```

    return nextClone;
  };
  const blurClone = prepareClone();
  const edgeClone = prepareClone();
  blurReplica.replaceChildren(blurClone);
  replica.replaceChildren(edgeClone);
  let frame = 0;
  const alignReplica = () => {
    const rect = pane.getBoundingClientRect();
    const blurPadding = 72;
    blurClone.style.position = "absolute";
    blurClone.style.left = `${-rect.left + blurPadding}px`;
    blurClone.style.top = `${-rect.top + blurPadding}px`;
    blurClone.style.width = `${window.innerWidth}px`;
    blurClone.style.height = `${window.innerHeight}px`;
    edgeClone.style.position = "absolute";
    edgeClone.style.left = `${-rect.left}px`;
    edgeClone.style.top = `${-rect.top}px`;
    edgeClone.style.width = `${window.innerWidth}px`;
    edgeClone.style.height = `${window.innerHeight}px`;
    edgeClone.style.transformOrigin = `${rect.left +
      rect.width / 2}px ${rect.top + rect.height / 2}px`;
    edgeClone.style.transform = "scale(1.090)";
    frame = window.requestAnimationFrame(alignReplica);
  };
  alignReplica();
  return () => {
    window.cancelAnimationFrame(frame);
    blurReplica.replaceChildren();
    replica.replaceChildren();
  };
}, []);
return (
  <div
    className={`liquid-glass-pane ${className}`}
    ref={paneRef}
    style={{ borderRadius: `${borderRadius}px` }}
  >
    <svg className="liquid-glass-pane__filter"
      aria-hidden="true">
      <defs>
        {filter ? (
          <filter id={filterId} x="-18%" y="-18%"
            width="136%" height="136%"
            colorInterpolationFilters="sRGB">
            <feGaussianBlur in="SourceGraphic"
              stdDeviation="24" result="blurredSource" />
            <felimage href={filter.displacement} x="0" y="0"
              width={filter.width} height={filter.height}
              result="displacementMap" />

```

```

    <feDisplacementMap
      in="blurredSource"
      in2="displacementMap"
      scale={filter.scale}
      xChannelSelector="R"
      yChannelSelector="G"
      result="displaced"
    />
    <feTurbulence
      type="fractalNoise"
      baseFrequency="0.009 0.024"
      numOctaves="1"
      seed="11"
      result="liquidRipple"
    />
    <feDisplacementMap
      in="displaced"
      in2="liquidRipple"
      scale="26"
      xChannelSelector="R"
      yChannelSelector="G"
      result="rippledGlass"
    />
    <feColorMatrix in="rippledGlass" type="saturate"
      values="3.4" result="saturated" />
    <feImage href={filter.specular} x="0" y="0"
      width={filter.width} height={filter.height}
      result="specularLayer" />
    <feComposite in="saturated" in2="specularLayer"
      operator="in" result="specularMask" />
    <feComponentTransfer in="specularLayer"
      result="fadedSpecular">
      <feFuncA type="linear" slope="0.58" />
    </feComponentTransfer>
    <feBlend in="specularMask" in2="displaced"
      mode="normal" result="saturatedGlass" />
    <feBlend in="fadedSpecular" in2="saturatedGlass"
      mode="normal" />
  </filter>
) : null}
</defs>
</svg>
<div className="liquid-glass-pane__blur-replica"
  ref={blurReplicaRef} aria-hidden="true" />
<div
  className="liquid-glass-pane__refraction"
  aria-hidden="true"
>
  <div className="liquid-glass-pane__replica"
    ref={replicaRef} />

```

```

</div>
<div className="liquid-glass-pane__cover"
  aria-hidden="true" />
<div className="liquid-glass-pane__tint"
  aria-hidden="true" />
<div className="liquid-glass-pane__dispersion"
  aria-hidden="true" />
<div className="liquid-glass-pane__reflect"
  aria-hidden="true" />
<div className="liquid-glass-pane__sharp"
  aria-hidden="true" />
<div className="liquid-glass-pane__content">{children}
</div>
</div>
);
}
// 源文件: packaging/windows/build.ps1
param(
  [switch]$Setup
)
$ErrorActionPreference = "Stop"
$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..\.."))
.Path
$BuildRoot = Join-Path $PSScriptRoot "build"
$DependencyRoot = Join-Path $PSScriptRoot "deps"
$PackageRoot = Join-Path $DependencyRoot "webview2"
$AppRoot = Join-Path $BuildRoot "Tomatotodo"
$PayloadStage = Join-Path $BuildRoot "payload\Tomatotodo"
$PayloadZip = Join-Path $BuildRoot "TomatotodoPayload.zip"
$WebViewVersion = "1.0.4078.44"
$AppVersion = "1.3.0"
$NugetPackage = Join-Path $DependencyRoot
  "Microsoft.Web.WebView2.$WebViewVersion.nupkg"
function Reset-Directory([string]$Path) {
  if (Test-Path -LiteralPath $Path) {
    Remove-Item -LiteralPath $Path -Recurse -Force
  }
  New-Item -ItemType Directory -Path $Path -Force | Out-Null
}
if (-not (Test-Path -LiteralPath (Join-Path $ProjectRoot
  "dist\client\index.html"))) {
  throw "dist/client/index.html is missing. Run pnpm run build
    first."
}
New-Item -ItemType Directory -Path $DependencyRoot -Force |
  Out-Null
if (-not (Test-Path -LiteralPath $NugetPackage)) {
  $Url = "https://api.nuget.org/v3-flatcontainer/microsoft.web.w
    ebview2/$WebViewVersion/microsoft.web.webview2.$WebViewVer
    sion.nupkg"

```

```

Invoke-WebRequest -Uri $Url -OutFile $NugetPackage
}
if (-not (Test-Path -LiteralPath (Join-Path $PackageRoot
    "lib\net462\Microsoft.Web.WebView2.Core.dll"))) {
    $ZipPath = Join-Path $DependencyRoot "webview2.zip"
    Copy-Item -LiteralPath $NugetPackage -Destination $ZipPath
        -Force
    Expand-Archive -LiteralPath $ZipPath -DestinationPath
        $PackageRoot -Force
    Remove-Item -LiteralPath $ZipPath -Force
}
Reset-Directory $AppRoot
New-Item -ItemType Directory -Path (Join-Path $AppRoot "www")
    -Force | Out-Null
$core = Join-Path $PackageRoot "lib\net462\Microsoft.Web.WebView
    2.Core.dll"
$WinForms = Join-Path $PackageRoot
    "lib\net462\Microsoft.Web.WebView2.WinForms.dll"
$Loader = Join-Path $PackageRoot
    "build\native\x64\WebView2Loader.dll"
$Icon = Join-Path $ProjectRoot "public\icons\Tomatotodo.ico"
$Compiler = Join-Path $env:WINDIR
    "Microsoft.NET\Framework64\v4.0.30319\csc.exe"
& $Compiler /nologo /target:winexe /platform:x64 /optimize+ `
    /win32manifest:"$PSScriptRoot\Tomatotodo.manifest" `
    /win32icon:"$Icon" `
    /reference:System.dll /reference:System.Core.dll
    /reference:System.Drawing.dll
    /reference:System.Windows.Forms.dll `
    /reference:"$Core" /reference:"$WinForms" `
    /out:"$AppRoot\Tomatotodo.exe" "$PSScriptRoot\Tomatotodo.cs"
if ($LASTEXITCODE -ne 0) { throw "Tomatotodo.exe compilation
    failed." }
Copy-Item -LiteralPath $Core,$WinForms,$Loader -Destination
    $AppRoot -Force
Copy-Item -LiteralPath (Join-Path $PackageRoot "LICENSE.txt")
    -Destination (Join-Path $AppRoot "WebView2-LICENSE.txt")
    -Force
Copy-Item -LiteralPath (Join-Path $PSScriptRoot
    "Tomatotodo.exe.config") -Destination $AppRoot -Force
Copy-Item -LiteralPath (Join-Path $PSScriptRoot
    "installer\Uninstall-Tomatotodo.ps1") -Destination $AppRoot
    -Force
Copy-Item -Path (Join-Path $ProjectRoot "dist\client\*")
    -Destination (Join-Path $AppRoot "www") -Recurse -Force
$Verify = Start-Process -FilePath (Join-Path $AppRoot
    "Tomatotodo.exe") -ArgumentList "--verify"
    -WorkingDirectory $AppRoot -PassThru -WindowStyle Hidden
$Verify.WaitForExit()
if ($Verify.ExitCode -ne 0) { throw "Native app verification

```

```

    failed with exit code $($Verify.ExitCode)." }
Write-Host "Windows app created: $AppRoot"
if ($Setup) {
    Reset-Directory $PayloadStage
    Copy-Item -Path (Join-Path $AppRoot "*") -Destination
        $PayloadStage -Recurse -Force
    Compress-Archive -Path $PayloadStage -DestinationPath
        $PayloadZip -CompressionLevel Optimal -Force
    $SetupPath = Join-Path $BuildRoot
        "Tomatotodo-Setup-$AppVersion.exe"
    & $Compiler /nologo /target:winexe /platform:x64 /optimize+ `
        /win32manifest:"$PSScriptRoot\Tomatotodo.manifest" `
        /win32icon:"$Icon" `
        /reference:System.dll /reference:System.Core.dll
        /reference:System.Windows.Forms.dll `
        /reference:System.IO.Compression.dll
        /reference:System.IO.Compression.FileSystem.dll
        /reference:Microsoft.CSharp.dll `
        "/resource:$PayloadZip,TomatotodoPayload.zip" `
        /out:"$SetupPath" "$PSScriptRoot\TomatotodoInstaller.cs"
    if ($LASTEXITCODE -ne 0) { throw "Setup compilation failed." }
    Write-Host "Single-file setup created: $SetupPath"
}
// 源文件: packaging/windows/installer/Install-Tomatotodo.ps1
param(
    [string]$PayloadPath = (Join-Path $PSScriptRoot
        "TomatotodoPayload.zip")
)
$ErrorActionPreference = "Stop"
$InstallRoot = Join-Path $env:LOCALAPPDATA "Programs\Tomatotodo"
$TempRoot = Join-Path $env:TEMP ("Tomatotodo-Install-" + [Guid]
    ::NewGuid().ToString("N"))
$DesktopShortcut = Join-Path ([Environment]
    ::GetFolderPath("Desktop")) "Tomatotodo.lnk"
$StartMenuFolder = Join-Path ([Environment]
    ::GetFolderPath("Programs")) "Tomatotodo"
$StartMenuShortcut = Join-Path $StartMenuFolder "Tomatotodo.lnk"
$UninstallShortcut = Join-Path $StartMenuFolder "卸载
    Tomatotodo.lnk"
try {
    Get-Process -Name "Tomatotodo" -ErrorAction SilentlyContinue
        | Stop-Process -Force
    New-Item -ItemType Directory -Path $TempRoot -Force | Out-Null
    Expand-Archive -LiteralPath $PayloadPath -DestinationPath
        $TempRoot -Force
    $SourceRoot = Join-Path $TempRoot "Tomatotodo"
    if (-not (Test-Path -LiteralPath (Join-Path $SourceRoot
        "Tomatotodo.exe"))) {
        throw "安装包内容不完整。"
    }
}

```

```
New-Item -ItemType Directory -Path $InstallRoot -Force |  
    Out-Null  
Copy-Item -Path (Join-Path $SourceRoot "*") -Destination  
    $InstallRoot -Recurse -Force  
New-Item -ItemType Directory -Path $StartMenuFolder -Force |  
    Out-Null  
$Shell = New-Object -ComObject WScript.Shell  
foreach ($ShortcutPath in @($DesktopShortcut,  
    $StartMenuShortcut)) {  
    $Shortcut = $Shell.CreateShortcut($ShortcutPath)  
    $Shortcut.TargetPath = Join-Path $InstallRoot  
        "Tomatotodo.exe"  
    $Shortcut.WorkingDirectory = $InstallRoot  
    $Shortcut.IconLocation = (Join-Path $InstallRoot  
        "Tomatotodo.exe") + ",0"  
    $Shortcut.Description = "Tomatotodo 番茄专注与任务清单"  
    $Shortcut.Save()  
}  
$Uninstall = $Shell.CreateShortcut($UninstallShortcut)  
$Uninstall.TargetPath = "$env:SystemRoot\System32\WindowsPower  
    Shell\v1.0\powershell.exe"  
$Uninstall.Arguments = '-NoProfile -ExecutionPolicy Bypass  
    -File "' + (Join-Path $InstallRoot  
        "Uninstall-Tomatotodo.ps1") + '"'  
$Uninstall.WorkingDirectory = $InstallRoot  
$Uninstall.IconLocation = (Join-Path $InstallRoot  
    "Tomatotodo.exe") + ",0"  
$Uninstall.Save()  
$UninstallKey = "HKCU:\Software\Microsoft\Windows\CurrentVersi  
    on\Uninstall\Tomatotodo"  
New-Item -Path $UninstallKey -Force | Out-Null  
$SizeKb = [math]::Ceiling(((Get-ChildItem -LiteralPath  
    $InstallRoot -Recurse -File | Measure-Object Length -Sum)  
    .Sum) / 1KB)  
New-ItemProperty -Path $UninstallKey -Name DisplayName -Value  
    "Tomatotodo" -PropertyType String -Force | Out-Null  
New-ItemProperty -Path $UninstallKey -Name DisplayVersion  
    -Value "1.3.0" -PropertyType String -Force | Out-Null  
New-ItemProperty -Path $UninstallKey -Name Publisher -Value  
    "Tomatotodo" -PropertyType String -Force | Out-Null  
New-ItemProperty -Path $UninstallKey -Name DisplayIcon -Value  
    (Join-Path $InstallRoot "Tomatotodo.exe") -PropertyType  
    String -Force | Out-Null  
New-ItemProperty -Path $UninstallKey -Name InstallLocation  
    -Value $InstallRoot -PropertyType String -Force | Out-Null  
New-ItemProperty -Path $UninstallKey -Name UninstallString  
    -Value ('powershell.exe -NoProfile -ExecutionPolicy  
    Bypass -File "' + (Join-Path $InstallRoot  
        "Uninstall-Tomatotodo.ps1") + '"') -PropertyType String  
    -Force | Out-Null
```

```

New-ItemProperty -Path $UninstallKey -Name EstimatedSize
-Value $SizeKb -PropertyType DWord -Force | Out-Null
New-ItemProperty -Path $UninstallKey -Name NoModify -Value 1
-PropertyType DWord -Force | Out-Null
New-ItemProperty -Path $UninstallKey -Name NoRepair -Value 1
-PropertyType DWord -Force | Out-Null
Start-Process -FilePath (Join-Path $InstallRoot
"Tomatotodo.exe") -WorkingDirectory $InstallRoot
Add-Type -AssemblyName System.Windows.Forms
[System.Windows.Forms.MessageBox]::Show("Tomatotodo 1.3.0
安装完成。", "Tomatotodo", "OK", "Information") | Out-Null
}
catch {
Add-Type -AssemblyName System.Windows.Forms
[System.Windows.Forms.MessageBox]
::Show("安装失败: $($_.Exception.Message)", "Tomatotodo",
"OK", "Error") | Out-Null
exit 1
}
finally {
if (Test-Path -LiteralPath $TempRoot) {
Remove-Item -LiteralPath $TempRoot -Recurse -Force
-ErrorAction SilentlyContinue
}
}
:: 源文件: packaging/windows/installer/install.cmd
@echo off
powershell.exe -NoProfile -ExecutionPolicy Bypass -File
"%~dp0Install-Tomatotodo.ps1" -PayloadPath
"%~dp0TomatotodoPayload.zip"
exit /b %errorlevel%
// 源文件: packaging/windows/installer/Uninstall-Tomatotodo.ps1
$ErrorActionPreference = "SilentlyContinue"
$InstallRoot = Split-Path -Parent $MyInvocation.MyCommand.Path
$DesktopShortcut = Join-Path ([Environment]
::GetFolderPath("Desktop")) "Tomatotodo.lnk"
$StartMenuFolder = Join-Path ([Environment]
::GetFolderPath("Programs")) "Tomatotodo"
Add-Type -AssemblyName System.Windows.Forms
$Choice = [System.Windows.Forms.MessageBox]::Show(
"确定要卸载 Tomatotodo
吗? 本地任务、便签和专注档案也会一并删除。",
"卸载 Tomatotodo",
"YesNo",
"Warning"
)
if ($Choice -ne "Yes") { exit 0 }
Get-Process -Name "Tomatotodo" -ErrorAction SilentlyContinue |
Stop-Process -Force
Remove-Item -LiteralPath $DesktopShortcut -Force -ErrorAction

```

```

    SilentlyContinue
Remove-Item -LiteralPath $StartMenuFolder -Recurse -Force
    -ErrorAction SilentlyContinue
Remove-Item -LiteralPath "HKCU:\Software\Microsoft\Windows\CurrentVersion\Uninstall\Tomatotodo" -Recurse -Force
    -ErrorAction SilentlyContinue
Remove-Item -LiteralPath (Join-Path $env:LOCALAPPDATA
    "Tomatotodo") -Recurse -Force -ErrorAction SilentlyContinue
$Cleanup = Join-Path $env:TEMP ("Tomatotodo-Cleanup-" + [Guid]
    ::NewGuid().ToString("N") + ".ps1")
$QuotedRoot = $InstallRoot.Replace('"', "'")
Set-Content -LiteralPath $Cleanup -Encoding UTF8 -Value
    "Start-Sleep -Milliseconds 800`nRemove-Item -LiteralPath
    '$QuotedRoot' -Recurse -Force -ErrorAction
    SilentlyContinue`nRemove-Item -LiteralPath
    '$MyInvocation.MyCommand.Path' -Force -ErrorAction
    SilentlyContinue"
Start-Process -FilePath "$env:SystemRoot\System32\WindowsPowerShell\v1.0\powershell.exe" -ArgumentList @("-NoProfile",
    "-ExecutionPolicy", "Bypass", "-File", $Cleanup)
    -WindowStyle Hidden
// 源文件: packaging/windows/Tomatotodo.cs
using System;
using System.Diagnostics;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Globalization;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Reflection;
using System.Runtime.InteropServices;
using System.Text;
using System.Threading;
using System.Windows.Forms;
using Microsoft.Web.WebView2.Core;
using Microsoft.Web.WebView2.WinForms;
using Microsoft.Win32;
[assembly: AssemblyTitle("Tomatotodo")]
[assembly: AssemblyDescription("Tomatotodo focus timer and task
    manager")]
[assembly: AssemblyCompany("Tomatotodo")]
[assembly: AssemblyProduct("Tomatotodo")]
[assembly: AssemblyVersion("1.3.0.0")]
[assembly: AssemblyFileVersion("1.3.0.0")]
namespace TomatotodoDesktop
{
    internal static class Program
    {
        private const string AppUrl = "http://localhost:4173/";
    }
}

```



```
private const string MutexName =
    "Local\\Tomatotodo-Desktop-1.0";
[STAThread]
private static int Main(string[] args)
{
    DpiAwareness.EnablePerMonitorV2();
    if (args.Length > 0 && string.Equals(args[0],
        "--verify", StringComparison.OrdinalIgnoreCase))
        return VerifyInstallation();
    bool ownsMutex;
    using (var mutex = new Mutex(true, MutexName, out
        ownsMutex))
    {
        if (!ownsMutex)
        {
            NativeMethods.ActivateExistingWindow();
            return 0;
        }
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        var webRoot = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "www");
        using (var server = new
            StaticFileServer(webRoot, 4173))
        {
            bool ownsServer = server.TryStart();
            if (!ownsServer &&
                !StaticFileServer.IsTomatotodoRunning(AppUrl))
            {
                MessageBox.Show("本地端口 4173
                    已被其他程序占用。", "Tomatotodo 无法启动",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
                return 2;
            }
            using (var window = new MainWindow(AppUrl))
            {
                Application.AddMessageFilter(new
                    FullscreenKeyFilter(window));
                Application.Run(window);
            }
        }
    }
    return 0;
}
private static int VerifyInstallation()
{
    try
    {

```

```

        string root = AppDomain.CurrentDomain.BaseDirectory;
        string[] required = {
            Path.Combine(root, "www", "index.html"),
            Path.Combine(root, "www",
                "manifest.webmanifest"),
            Path.Combine(root,
                "Microsoft.Web.WebView2.Core.dll"),
            Path.Combine(root,
                "Microsoft.Web.WebView2.WinForms.dll"),
            Path.Combine(root, "WebView2Loader.dll")
        };
        foreach (string path in required) if
            (!File.Exists(path)) return 10;
        return string.IsNullOrEmpty(CoreWebView2Environment
            .GetAvailableBrowserVersionString())
            ? 11 : 0;
    }
    catch { return 12; }
}

internal sealed class FullscreenKeyFilter : IMessageFilter
{
    private const int WmKeyDown = 0x0100;
    private const int WmSysKeyDown = 0x0104;
    private const int VkF12 = 0x7B;
    private readonly MainWindow window;
    private bool keyDown;
    public FullscreenKeyFilter(MainWindow window) {
        this.window = window; }
    public bool PreFilterMessage(ref Message message)
    {
        if ((message.Msg == WmKeyDown || message.Msg ==
            WmSysKeyDown) && message.WParam.ToInt32() ==
            VkF12)
        {
            if (!keyDown)
            {
                keyDown = true;
                window.BeginInvoke(new
                    Action(window.ToggleFullscreen));
            }
            return true;
        }
        if (message.Msg == 0x0101 &&
            message.WParam.ToInt32() == VkF12) keyDown =
            false;
        return false;
    }
}

```

```
internal sealed class MainWindow : Form
{
    private readonly WebView2 webView;
    private readonly string appUrl;
    private readonly MiniTimerWindow miniTimer;
    private bool fullscreen;
    private FormBorderStyle savedBorder;
    private FormWindowState savedState;
    private Rectangle savedBounds;
    private bool savedTopMost;
    private IntPtr keyboardHook;
    private NativeMethods.LowLevelKeyboardProc keyboardProc;
    private bool f12Down;
    public MainWindow(string appUrl)
    {
        this.appUrl = appUrl;
        Text = "Tomatotodo";
        StartPosition = FormStartPosition.CenterScreen;
        WindowState = FormWindowState.Maximized;
        MinimumSize = new Size(1024, 640);
        AutoScaleDimensions = new.SizeF(96F, 96F);
        AutoScaleMode = AutoScaleMode.Dpi;
        BackColor = Color.FromArgb(11, 13, 11);
        Icon = Icon.ExtractAssociatedIcon(Application.ExecutablePath);
        miniTimer = new MiniTimerWindow(Icon);
        miniTimer.UserHidden += delegate {
            try { webView.CoreWebView2.PostWebMessageAsString("mini-window-hidden"); } catch { }
        };
        miniTimer.ToggleRequested += delegate {
            try { webView.CoreWebView2.PostWebMessageAsString("mini-window-toggle"); } catch { }
        };
        webView = new WebView2 {
            Dock = DockStyle.Fill,
            DefaultBackgroundColor = Color.FromArgb(11, 13, 11)
        };
        Controls.Add(webView);
        Shown += InitializeWebView;
        HandleCreated += InstallKeyboardHook;
        HandleCreated += ApplySystemTitleBarTheme;
        FormClosed += RemoveKeyboardHook;
        FormClosed += delegate { miniTimer.Close(); };
    }
    private void ApplySystemTitleBarTheme(object sender, EventArgs e)
    {
        ApplyAppTheme(NativeWindowTheme.SystemUsesDarkApps())
    }
}
```

```
        );
    }
    private void ApplyAppTheme(bool useDarkMode)
    {
        Color background = useDarkMode ? Color.FromArgb(11,
            16, 12) : Color.FromArgb(248, 246, 238);
        BackColor = background;
        webView.DefaultBackgroundColor = background;
        NativeWindowTheme.Apply(Handle, useDarkMode);
    }
    private void InstallKeyboardHook(object sender,
        EventArgs e)
    {
        if (keyboardHook != IntPtr.Zero) return;
        keyboardProc = HandleLowLevelKey;
        keyboardHook = NativeMethods.SetWindowsHookEx(13,
            keyboardProc, IntPtr.Zero, 0);
    }
    private void RemoveKeyboardHook(object sender,
        FormClosedEventArgs e)
    {
        if (keyboardHook == IntPtr.Zero) return;
        NativeMethods.UnhookWindowsHookEx(keyboardHook);
        keyboardHook = IntPtr.Zero;
    }
    private IntPtr HandleLowLevelKey(int code, IntPtr
        message, IntPtr data)
    {
        if (code >= 0 && NativeMethods.IsAppForeground(Handle
            e))
        {
            int virtualKey = Marshal.ReadInt32(data);
            int keyMessage = message.ToInt32();
            if (virtualKey == 0x7B)
            {
                {
                    if ((keyMessage == 0x0100 || keyMessage ==
                        0x0104) && !f12Down)
                    {
                        f12Down = true;
                        BeginInvoke(new Action(ToggleFullscreen)
                            );
                    }
                    if (keyMessage == 0x0101 || keyMessage ==
                        0x0105) f12Down = false;
                    return new IntPtr(1);
                }
            }
        }
        return NativeMethods.CallNextHookEx(keyboardHook,
            code, message, data);
    }
}
```

```
private async void InitializeWebView(object sender,
    EventArgs e)
{
    try
    {
        string userData =
            Path.Combine(Environment.GetFolderPath(Envir
                onment.SpecialFolder.LocalApplicationData),
                "Tomatotodo", "WebView2");
        Directory.CreateDirectory(userData);
        var environment = await
            CoreWebView2Environment.CreateAsync(null,
                userData);
        await webView.EnsureCoreWebView2Async(enviromen
            t);
        webView.CoreWebView2.Settings.AreDevToolsEnabled
            = false;
        webView.CoreWebView2.Settings.IsStatusBarEnabled
            = false;
        webView.CoreWebView2.Settings.IsZoomControlEnabl
            ed = false;
        webView.CoreWebView2.Settings.AreBrowserAcceler
            atorKeysEnabled = false;
        webView.ZoomFactor = 1.0;
        webView.CoreWebView2.NewWindowRequested +=
            HandleNewWindow;
        webView.CoreWebView2.WebMessageReceived +=
            HandleWebMessage;
        webView.CoreWebView2.PermissionRequested +=
            HandlePermissionRequested;
        await webView.CoreWebView2.AddScriptToExecuteOnD
            ocumentCreatedAsync(
                "window.addEventListener('keydown',
                function(event){"
                + "if(event.key==='F12')
                {event.preventDefault();
                event.stopImmediatePropagation();"
                + "window.chrome.webview.postMessage('toggle
                -fullscreen');}},true);");
        webView.CoreWebView2.Navigate(appUrl);
    }
    catch (Exception exception)
    {
        MessageBox.Show("Tomatotodo 无法载入
            WebView2。 \n\n" + exception.Message,
            "Tomatotodo 无法启动", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        Close();
    }
}
```

```
private void HandlePermissionRequested(object sender,
    CoreWebView2PermissionRequestedEventArgs e)
{
    if (e.PermissionKind !=
        CoreWebView2PermissionKind.FileReadWrite ||
        !e.Uri.StartsWith(appUrl,
            StringComparison.OrdinalIgnoreCase)) return;
    e.State = CoreWebView2PermissionState.Allow;
    e.SavesInProfile = true;
    e.Handled = true;
}

public void ToggleFullscreen()
{
    if (!fullscreen)
    {
        savedBorder = FormBorderStyle;
        savedState = WindowState;
        savedBounds = Bounds;
        savedTopMost = TopMost;
        SuspendLayout();
        WindowState = FormWindowState.Normal;
        FormBorderStyle = FormBorderStyle.None;
        TopMost = true;
        Bounds = Screen.FromControl(this).Bounds;
        ResumeLayout(true);
        fullscreen = true;
        return;
    }
    SuspendLayout();
    TopMost = savedTopMost;
    FormBorderStyle = savedBorder;
    Bounds = savedBounds;
    WindowState = savedState;
    ResumeLayout(true);
    fullscreen = false;
}

private void HandleWebMessage(object sender,
    CoreWebView2WebMessageReceivedEventArgs e)
{
    try
    {
        string message = e.TryGetWebMessageAsString();
        if (message == "toggle-fullscreen")
            BeginInvoke(new Action(ToggleFullscreen));
        if (message == "app-theme | dark")
            BeginInvoke(new Action(() =>
                ApplyAppTheme(true)));
        if (message == "app-theme | light")
            BeginInvoke(new Action(() =>
                ApplyAppTheme(false)));
    }
}
```

```

        if (message == "mini-window|hide")
            BeginInvoke(new Action(miniTimer.Hide));
        if (message.StartsWith("mini-window|show|",
            StringComparison.Ordinal) ||
            message.StartsWith("mini-window|update|",
            StringComparison.Ordinal))
        {
            string[] parts = message.Split(new[] { '|' }
            , 6);
            if (parts.Length >= 5)
            {
                string phase = parts[2];
                string status = parts[3];
                string time = parts[4];
                double progress = 0;
                if (parts.Length >= 6)
                double.TryParse(parts[5],
                NumberStyles.Float,
                CultureInfo.InvariantCulture, out progress);
                BeginInvoke(new Action(() => {
                    miniTimer.UpdateTimer(phase, status,
                    time, progress);
                    if (parts[1] == "show" &&
                    !miniTimer.Visible) miniTimer.Show();
                }));
            }
        }
    }
    catch { }
}

private void HandleNewWindow(object sender,
    CoreWebView2NewWindowRequestedEventArgs e)
{
    e.Handled = true;
    try { Process.Start(e.Uri); } catch { }
}

}

internal sealed class MiniTimerWindow : Form
{
    private readonly Label phaseLabel;
    private readonly MiniProgressButton statusLabel;
    private readonly Label timeLabel;
    public event EventHandler UserHidden;
    public event EventHandler ToggleRequested;
    public MiniTimerWindow(Icon icon)
    {
        Text = "Tomatotodo 小窗";
        Icon = icon;
        Width = 320;
        Height = 174;
    }
}

```

```
MinimumSize = new Size(260, 148);
FormBorderStyle = FormBorderStyle.SizableToolWindow;
StartPosition = FormStartPosition.Manual;
ShowInTaskbar = false;
TopMost = true;
AutoScaleMode = AutoScaleMode.Dpi;
BackColor = Color.FromArgb(15, 31, 20);
Padding = new Padding(22, 16, 22, 15);
phaseLabel = new Label {
    AutoSize = true,
    ForeColor = Color.FromArgb(157, 196, 151),
    Font = new Font("Microsoft YaHei UI", 11F,
        FontStyle.Bold)
};
statusLabel = new MiniProgressButton {
    Width = 70,
    Height = 24,
    ForeColor = Color.FromArgb(170, 184, 173),
    Font = new Font("Microsoft YaHei UI", 8.5F),
    Cursor = Cursors.Hand
};
statusLabel.Click += delegate {
    if (ToggleRequested != null)
        ToggleRequested(this, EventArgs.Empty);
};
timeLabel = new Label {
    AutoSize = true,
    ForeColor = Color.FromArgb(248, 246, 238),
    Font = new Font("Georgia", 34F,
        FontStyle.Regular)
};
var heading = new TableLayoutPanel { Dock =
    DockStyle.Top, Height = 28, ColumnCount = 2 };
heading.ColumnStyles.Add(new
    ColumnStyle(SizeType.Percent, 55));
heading.ColumnStyles.Add(new
    ColumnStyle(SizeType.Percent, 45));
heading.Controls.Add(phaseLabel, 0, 0);
heading.Controls.Add(statusLabel, 1, 0);
statusLabel.Anchor = AnchorStyles.Top |
    AnchorStyles.Right;
var timePanel = new Panel { Dock = DockStyle.Fill };
timePanel.Controls.Add(timeLabel);
timeLabel.Location = new Point(0, 16);
Controls.Add(timePanel);
Controls.Add(heading);
FormClosing += delegate(object sender,
    FormClosingEventArgs e) {
    if (e.CloseReason == CloseReason.UserClosing) {
        e.Cancel = true;
    }
}
```



```

        Hide();
        if (UserHidden != null) UserHidden(this,
            EventArgs.Empty);
    }
};
}
protected override void OnShown(EventArgs e)
{
    base.OnShown(e);
    Rectangle area = Screen.PrimaryScreen.WorkingArea;
    Location = new Point(area.Right - Width - 22,
        area.Bottom - Height - 22);
}
public void UpdateTimer(string phase, string status,
    string time, double progress)
{
    phaseLabel.Text = phase;
    statusLabel.Text = status;
    statusLabel.Progress = Math.Max(0, Math.Min(1,
        progress / 100.0));
    timeLabel.Text = time;
}
}
internal sealed class MiniProgressButton : Control
{
    private double progress;
    private bool hovered;
    public double Progress
    {
        get { return progress; }
        set { progress = value; Invalidate(); }
    }
    public MiniProgressButton()
    {
        DoubleBuffered = true;
        SetStyle(ControlStyles.UserPaint |
            ControlStyles.AllPaintingInWmPaint |
            ControlStyles.OptimizedDoubleBuffer, true);
        MouseEnter += delegate { hovered = true;
            Invalidate(); };
        MouseLeave += delegate { hovered = false;
            Invalidate(); };
    }
    protected override void OnPaint(PaintEventArgs e)
    {
        base.OnPaint(e);
        e.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
        RectangleF bounds = new RectangleF(0.5F, 0.5F,
            Width - 1F, Height - 1F);
        float radius = Height / 2F;

```

```

using (GraphicsPath track = RoundedPath(bounds,
    radius))
using (SolidBrush trackBrush = new
    SolidBrush(Color.FromArgb(18, 245, 246, 239)))
using (Pen border = new Pen(Color.FromArgb(hovered
    ? 70 : 38, 124, 169, 130)))
{
    e.Graphics.FillPath(trackBrush, track);
    if (Progress > 0)
    {
        e.Graphics.SetClip(track);
        using (SolidBrush fill = new
            SolidBrush(Color.FromArgb(hovered ? 48 : 34,
                124, 169, 130)))
        {
            e.Graphics.FillRectangle(fill, 0, 0,
                (float)(Width * Progress), Height);
            e.Graphics.ResetClip();
        }
        e.Graphics.DrawPath(border, track);
    }
    TextRenderer.DrawText(e.Graphics, Text, Font,
        ClientRectangle, hovered ? Color.FromArgb(232,
            239, 229) : ForeColor,
        TextFormatFlags.HorizontalCenter |
            TextFormatFlags.VerticalCenter |
            TextFormatFlags.NoPadding);
}
private static GraphicsPath RoundedPath(RectangleF
    rectangle, float radius)
{
    float diameter = radius * 2;
    GraphicsPath path = new GraphicsPath();
    path.AddArc(rectangle.Left, rectangle.Top, diameter,
        diameter, 180, 90);
    path.AddArc(rectangle.Right - diameter,
        rectangle.Top, diameter, diameter, 270, 90);
    path.AddArc(rectangle.Right - diameter,
        rectangle.Bottom - diameter, diameter, diameter,
        0, 90);
    path.AddArc(rectangle.Left, rectangle.Bottom -
        diameter, diameter, diameter, 90, 90);
    path.CloseFigure();
    return path;
}
}
internal static class DpiAwareness
{
    private static readonly IntPtr PerMonitorV2 = new
        IntPtr(-4);
    public static void EnablePerMonitorV2()

```

```

{
    try { if (SetProcessDpiAwarenessContext(PerMonitorV2
        )) return; } catch { }
    try { SetProcessDpiAwareness(2); } catch { }
    try { SetProcessDPIAware(); } catch { }
}
[DllImport("user32.dll")] private static extern bool
    SetProcessDpiAwarenessContext(IntPtr value);
[DllImport("shcore.dll")] private static extern int
    SetProcessDpiAwareness(int awareness);
[DllImport("user32.dll")] private static extern bool
    SetProcessDPIAware();
}
internal static class NativeWindowTheme
{
    private const int ImmersiveDarkModeBefore20H1 = 19;
    private const int ImmersiveDarkMode = 20;
    private const int CaptionColor = 35;
    private const int TextColor = 36;
    public static void Apply(IntPtr window)
    {
        Apply(window, SystemUsesDarkApps());
    }
    public static void Apply(IntPtr window, bool
        useDarkMode)
    {
        int dark = useDarkMode ? 1 : 0;
        try
        {
            if (DwmSetWindowAttribute(window,
                ImmersiveDarkMode, ref dark, sizeof(int))
                != 0)
                DwmSetWindowAttribute(window,
                    ImmersiveDarkModeBefore20H1, ref dark,
                    sizeof(int));
            int caption = ColorTranslator.ToWin32(useDarkMod
                e ? Color.FromArgb(24, 31, 26) :
                Color.FromArgb(248, 246, 238));
            int text = ColorTranslator.ToWin32(useDarkMode
                ? Color.FromArgb(232, 238, 232) :
                Color.FromArgb(35, 40, 36));
            DwmSetWindowAttribute(window, CaptionColor, ref
                caption, sizeof(int));
            DwmSetWindowAttribute(window, TextColor, ref
                text, sizeof(int));
        }
        catch { }
    }
}
public static bool SystemUsesDarkApps()
{

```

```

try
{
    using (RegistryKey key =
        Registry.CurrentUser.OpenSubKey(@"Software\Microsoft\Windows\CurrentVersion\Themes\Personalize"))
    {
        object value = key == null ? null :
            key.GetValue("AppsUseLightTheme");
        return value is int && (int)value == 0;
    }
}
catch { return true; }
}
[DllImport("dwmapi.dll")]
private static extern int DwmSetWindowAttribute(IntPtr
    window, int attribute, ref int value, int valueSize)
    ;
}
internal sealed class StaticFileServer : IDisposable
{
    private readonly string root;
    private readonly int port;
    private TcpListener listener;
    private volatile bool running;
    public StaticFileServer(string root, int port)
    {
        this.root = Path.GetFullPath(root)
            .TrimEnd(Path.DirectorySeparatorChar) +
            Path.DirectorySeparatorChar;
        this.port = port;
    }
    public bool TryStart()
    {
        try
        {
            listener = new TcpListener(IPAddress.Loopback,
                port);
            listener.Start();
            running = true;
            var thread = new Thread(ListenLoop) {
                IsBackground = true, Name = "Tomatotodo
                static server" };
            thread.Start();
            return true;
        }
        catch (SocketException) { return false; }
    }
    public static bool IsTomatotodoRunning(string appUrl)
    {

```

```
try
{
    using (var client = new WebClient())
        return client.DownloadString(new Uri(new
            Uri(appUrl), "manifest.webmanifest"))
            .IndexOf("Tomatotodo",
                StringComparison.OrdinalIgnoreCase) >= 0;
}
catch { return false; }
}

private void ListenLoop()
{
    while (running)
    {
        try { ThreadPool.QueueUserWorkItem(HandleClient,
            listener.AcceptTcpClient()); }
        catch (SocketException) { if (!running) return; }
        catch (ObjectDisposedException) { return; }
    }
}

private void HandleClient(object state)
{
    using (var client = (TcpClient)state)
    {
        client.ReceiveTimeout = 5000;
        client.SendTimeout = 5000;
        NetworkStream stream = client.GetStream();
        try
        {
            string line;
            using (var reader = new StreamReader(stream,
                Encoding.ASCII, false, 1024, true))
            {
                line = reader.ReadLine();
                string header;
                do { header = reader.ReadLine(); }
                while (!string.IsNullOrEmpty(header));
            }
            if (string.IsNullOrEmptyOrWhiteSpace(line)) return;
            string[] request = line.Split(' ');
            if (request.Length < 2 || (request[0] !=
                "GET" && request[0] != "HEAD")) return;
            string path = ResolvePath(request[1]);
            if (path == null) return;
            if (!File.Exists(path)) path =
                Path.Combine(root, "index.html");
            byte[] body = File.ReadAllBytes(path);
            string name = Path.GetFileName(path);
            bool noCache = name == "index.html" || name
```

```

        == "sw.js" || name ==
        "manifest.webmanifest";
        string headers = "HTTP/1.1 200
        OK\r\nContent-Type: " + ContentType(path) +
        "\r\nContent-Length: " + body.Length
            + "\r\nCache-Control: " + (noCache ?
        "no-cache" : "public, max-age=31536000,
        immutable") + "\r\nConnection:
        close\r\n\r\n";
        byte[] headerBytes =
        Encoding.ASCII.GetBytes(headers);
        stream.Write(headerBytes, 0,
        headerBytes.Length);
        if (request[0] == "GET") stream.Write(body,
        0, body.Length);
    }
    catch { }
}
}
private string ResolvePath(string target)
{
    string decoded = Uri.UnescapeDataString(target.Split
        ('?')[0]).TrimStart('/');
    if (string.IsNullOrEmpty(decoded) ||
        decoded.EndsWith("/")) decoded += "index.html";
    string candidate = Path.GetFullPath(Path.Combine(roo
        t, decoded.Replace('/',
        Path.DirectorySeparatorChar)));
    return candidate.StartsWith(root,
        StringComparison.OrdinalIgnoreCase) ? candidate
        : null;
}
private static string ContentType(string path)
{
    switch (Path.GetExtension(path).ToLowerInvariant())
    {
        case ".css": return "text/css; charset=utf-8";
        case ".html": return "text/html; charset=utf-8";
        case ".ico": return "image/x-icon";
        case ".js": return "text/javascript;
            charset=utf-8";
        case ".json": return "application/json;
            charset=utf-8";
        case ".png": return "image/png";
        case ".svg": return "image/svg+xml";
        case ".webmanifest": return
            "application/manifest+json; charset=utf-8";
        case ".webp": return "image/webp";
        default: return "application/octet-stream";
    }
}

```

```

    }
    public void Dispose() { running = false; if (listener
        != null) listener.Stop(); }
}
internal static class NativeMethods
{
    public delegate IntPtr LowLevelKeyboardProc(int code,
        IntPtr message, IntPtr data);
    [DllImport("user32.dll", SetLastError = true)] private
        static extern IntPtr FindWindow(string className,
            string windowName);
    [DllImport("user32.dll")] private static extern bool
        SetForegroundWindow(IntPtr window);
    [DllImport("user32.dll")] private static extern bool
        ShowWindow(IntPtr window, int command);
    [DllImport("user32.dll")] private static extern IntPtr
        GetForegroundWindow();
    [DllImport("user32.dll")] private static extern IntPtr
        GetAncestor(IntPtr window, uint flags);
    [DllImport("user32.dll", SetLastError = true)] public
        static extern IntPtr SetWindowsHookEx(int hook,
            LowLevelKeyboardProc callback, IntPtr module, uint
            threadId);
    [DllImport("user32.dll", SetLastError = true)] public
        static extern bool UnhookWindowsHookEx(IntPtr hook);
    [DllImport("user32.dll")] public static extern IntPtr
        CallNextHookEx(IntPtr hook, int code, IntPtr
            message, IntPtr data);
    public static bool IsAppForeground(IntPtr appWindow)
    {
        IntPtr foreground = GetForegroundWindow();
        return foreground == appWindow ||
            GetAncestor(foreground, 2) == appWindow;
    }
    public static void ActivateExistingWindow()
    {
        IntPtr window = FindWindow(null, "Tomatotodo");
        if (window == IntPtr.Zero) return;
        ShowWindow(window, 9);
        SetForegroundWindow(window);
    }
}
}
<!-- 源文件: packaging/windows/Tomatotodo.exe.config -->
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
    <startup useLegacyV2RuntimeActivationPolicy="true">
        <supportedRuntime version="v4.0" sku=".NETFramework,
            Version=v4.8" />
    </startup>

```

```

</configuration>
<!-- 源文件: packaging/windows/Tomatotodo.manifest -->
<?xml version="1.0" encoding="utf-8"?>
<assembly manifestVersion="1.0" xmlns="urn:schemas-microsoft-com
    :asm.v1">
    <assemblyIdentity version="1.3.0.0" name="Tomatotodo.Desktop"
        />
    <description>Tomatotodo</description>
    <dependency>
        <dependentAssembly>
            <assemblyIdentity type="win32"
                name="Microsoft.Windows.Common-Controls"
                version="6.0.0.0" processorArchitecture=""
                publicKeyToken="6595b64144ccf1df" language="" />
        </dependentAssembly>
    </dependency>
    <application xmlns="urn:schemas-microsoft-com:asm.v3">
        <windowsSettings>
            <dpiAware xmlns="http://schemas.microsoft.com/SMI/2005/Win
                dowsSettings">true/pm</dpiAware>
            <dpiAwareness xmlns="http://schemas.microsoft.com/SMI/2016
                /WindowsSettings">PerMonitorV2,
                PerMonitor</dpiAwareness>
            <longPathAware xmlns="http://schemas.microsoft.com/SMI/201
                6/WindowsSettings">true</longPathAware>
        </windowsSettings>
    </application>
</assembly>
// 源文件: packaging/windows/TomatotodoInstaller.cs
using System;
using System.Diagnostics;
using System.IO;
using System.IO.Compression;
using System.Reflection;
using System.Windows.Forms;
using Microsoft.Win32;
[assembly: AssemblyTitle("Tomatotodo Setup")]
[assembly: AssemblyDescription("Tomatotodo 1.3.0
    番茄专注与任务管理安装程序")]
[assembly: AssemblyCompany("Tomatotodo")]
[assembly: AssemblyProduct("Tomatotodo")]
[assembly: AssemblyVersion("1.3.0.0")]
[assembly: AssemblyFileVersion("1.3.0.0")]
namespace TomatotodoSetup
{
    internal static class Installer
    {
        private const string ResourceName =
            "TomatotodoPayload.zip";
        [STAThread]

```



```
private static int Main()
{
    DpiAwareness.EnablePerMonitorV2();
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false)
        ;
    if (MessageBox.Show(
        "安装 Tomatotodo
        1.3.0\n\n 安装范围：当前用户\n 快捷方式：桌面
        、开始菜单\n 升级安装会保留本地任务、设置与专
        注记录。 \n\n 是否继续？ ",
        "安装 Tomatotodo",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Information) != DialogResult.Yes)
        return 0;
    string installRoot =
        Path.Combine(Environment.GetFolderPath(Environme
        nt.SpecialFolder.LocalApplicationData),
        "Programs", "Tomatotodo");
    string tempRoot = Path.Combine(Path.GetTempPath(),
        "Tomatotodo-Install-" + Guid.NewGuid()
        .ToString("N"));
    try
    {
        StopRunningApp();
        Directory.CreateDirectory(tempRoot);
        string payloadPath = Path.Combine(tempRoot,
            "payload.zip");
        ExtractPayloadResource(payloadPath);
        ZipFile.ExtractToDirectory(payloadPath,
            tempRoot);
        string sourceRoot = Path.Combine(tempRoot,
            "Tomatotodo");
        if (!File.Exists(Path.Combine(sourceRoot,
            "Tomatotodo.exe"))) throw new
            InvalidDataException("安装包内容不完整。");
        Directory.CreateDirectory(installRoot);
        CopyDirectory(sourceRoot, installRoot);
        CreateShortcuts(installRoot);
        RegisterUninstaller(installRoot);
        MessageBox.Show("Tomatotodo 1.3.0
            安装完成。 \n\n 按 F12 可进入或退出全屏。",
            "安装完成", MessageBoxButtons.OK,
            MessageBoxIcon.Information);
        Process.Start(new ProcessStartInfo { FileName =
            Path.Combine(installRoot, "Tomatotodo.exe"),
            WorkingDirectory = installRoot,
            UseShellExecute = true });
        return 0;
    }
}
```

```
catch (Exception exception)
{
    MessageBox.Show("未能完成安装。 \n\n" +
        exception.Message, "Tomatotodo",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    return 1;
}
finally
{
    try { if (Directory.Exists(tempRoot))
        Directory.Delete(tempRoot, true); } catch {
    }
}
}

private static void ExtractPayloadResource(string
    destination)
{
    using (Stream input =
        Assembly.GetExecutingAssembly()
        .GetManifestResourceStream(ResourceName))
    {
        if (input == null) throw new
            InvalidDataException("找不到安装资源。");
        using (FileStream output =
            File.Create(destination))
            input.CopyTo(output);
    }
}

private static void StopRunningApp()
{
    foreach (Process process in
        Process.GetProcessesByName("Tomatotodo"))
    {
        try { process.CloseMainWindow(); if
            (!process.WaitForExit(1500)) process.Kill();
            } catch { } finally { process.Dispose(); }
    }
}

private static void CopyDirectory(string source, string
    destination)
{
    Directory.CreateDirectory(destination);
    foreach (string file in Directory.GetFiles(source))
        File.Copy(file, Path.Combine(destination,
            Path.GetFileName(file)), true);
    foreach (string directory in
        Directory.GetDirectories(source))
        CopyDirectory(directory,
            Path.Combine(destination,
                Path.GetFileName(directory)));
}
```

```
}  
private static void CreateShortcuts(string installRoot)  
{  
    string startMenu = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.Programs),  
        "Tomatotodo");  
    Directory.CreateDirectory(startMenu);  
    Type shellType = Type.GetTypeFromProgID("WScript.Shell");  
    if (shellType == null) throw new  
        InvalidOperationException("无法创建 Windows  
        快捷方式。");  
    dynamic shell = Activator.CreateInstance(shellType);  
    string executable = Path.Combine(installRoot,  
        "Tomatotodo.exe");  
    CreateShortcut(shell,  
        Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.DesktopDirectory),  
            "Tomatotodo.lnk"), executable, "", installRoot,  
            executable + ",0");  
    CreateShortcut(shell, Path.Combine(startMenu,  
        "Tomatotodo.lnk"), executable, "", installRoot,  
            executable + ",0");  
    string powershell = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.System),  
        "WindowsPowerShell", "v1.0", "powershell.exe");  
    CreateShortcut(shell, Path.Combine(startMenu, "卸载  
        Tomatotodo.lnk"), powershell, "-NoProfile  
        -ExecutionPolicy Bypass -File \"" +  
        Path.Combine(installRoot,  
            "Uninstall-Tomatotodo.ps1") + "\"", installRoot,  
            executable + ",0");  
}  
private static void CreateShortcut(dynamic shell,  
    string path, string target, string arguments,  
    string workingDirectory, string icon)  
{  
    dynamic shortcut = shell.CreateShortcut(path);  
    shortcut.TargetPath = target;  
    shortcut.Arguments = arguments;  
    shortcut.WorkingDirectory = workingDirectory;  
    shortcut.IconLocation = icon;  
    shortcut.Description = "Tomatotodo  
        番茄专注与任务清单";  
    shortcut.Save();  
}  
private static void RegisterUninstaller(string  
    installRoot)  
{  
    using (RegistryKey key =
```

```

Registry.CurrentUser.CreateSubKey(@"Software\Microsoft\Windows\CurrentVersion\Uninstall\Tomatotodo"))
{
    if (key == null) throw new
        InvalidOperationException("无法写入卸载信息
        。");
    string executable = Path.Combine(installRoot,
        "Tomatotodo.exe");
    key.SetValue("DisplayName", "Tomatotodo",
        RegistryValueKind.String);
    key.SetValue("DisplayVersion", "1.3.0",
        RegistryValueKind.String);
    key.SetValue("Publisher", "Tomatotodo",
        RegistryValueKind.String);
    key.SetValue("DisplayIcon", executable,
        RegistryValueKind.String);
    key.SetValue("InstallLocation", installRoot,
        RegistryValueKind.String);
    key.SetValue("UninstallString", "powershell.exe
        -NoProfile -ExecutionPolicy Bypass -File
        \"\" + Path.Combine(installRoot,
        "Uninstall-Tomatotodo.ps1") + "\"",
        RegistryValueKind.String);
    key.SetValue("NoModify", 1,
        RegistryValueKind.DWord);
    key.SetValue("NoRepair", 1,
        RegistryValueKind.DWord);
}
}
}
internal static class DpiAwareness
{
    [System.Runtime.InteropServices.DllImport("user32.dll")]
    private static extern bool
        SetProcessDpiAwarenessContext(IntPtr value);
    [System.Runtime.InteropServices.DllImport("shcore.dll")]
    private static extern int
        SetProcessDpiAwareness(int value);
    [System.Runtime.InteropServices.DllImport("user32.dll")]
    private static extern bool SetProcessDPIAware();
    public static void EnablePerMonitorV2()
    {
        try { if (SetProcessDpiAwarenessContext(new
            IntPtr(-4))) return; } catch { }
        try { SetProcessDpiAwareness(2); } catch { }
        try { SetProcessDPIAware(); } catch { }
    }
}
}

```